Electoral Equilibrium

A Stochastic Optimization Framework for Key Voter Groups

Seven-Week Development Plan

Research Assistant: Juhi Damley **Supervising Mentor:** Prof. Gaston Espinosa
Claremont McKenna College *May 2026*

The Big Idea

Electoral forecasting today is static. Polling aggregators like FiveThirtyEight tell you where the race stands right now — but they cannot answer the question every campaign strategist asks: *"If **X** happens tomorrow, how does that change which voter groups we need to win?"*

Electoral Equilibrium answers that question in real time. A user selects a party (Democrat or Republican), types any hypothetical political event — an assassination attempt, a financial scandal, an October Surprise — and the system immediately shows how that party's coalition must rebalance across the key demographic groups (using a three-layer race / religion / gender hierarchy) to maintain a winning position, along with the probability of achieving it. The demographic architecture uses a parallel three-stratum architecture: Race/Ethnicity, Religion, and Gender each cover ∼100% of the electorate as independent strata. Every voter belongs to exactly one cell in each stratum, and effective loyalty is the weighted sum across all three — no nested cross-tabulations, no sparse cells. White voters and men are included as explicit cells alongside all other groups.

The system is built on three technical contributions: (1) a fine-tuned Mistral 7B language model trained on 25+ historical shock events to estimate how each group's loyalty shifts; (2) a mean-variance stochastic optimizer (inspired by portfolio theory) that rebalances the coalition under the shifted estimates; and (3) a two-stage NLP pipeline using RoBERTa and bio-based demographic inference from social media to construct the training data. The result is a live, publicly accessible web application with a password-protected analyst dashboard — grounded in two decades of Prof. Gaston Espinosa's published research on religion, race, and presidential voting.

1 Project Philosophy and Engineering Principles

This development plan implements the research pipeline specified in the SRP proposal, extended to include a two-stage NLP architecture, a social media sentiment layer, and a fine-tuned LLM for hypothetical shock estimation:

Data → Baseline → News + Social Media → RoBERTa → LLM Fine-Tune → Shock Estimation → Stochastic Optimization → Web App

The two-stage NLP design is a core methodological contribution. Social media sentiment (Bluesky, Apify, Facebook, Truth Social, Telegram) runs in parallel with the news corpus scraper, providing a

high-frequency weak supervisory signal that addresses the undercoverage problem inherent in having only 6–8 historical election cycles. Platform user-base demographics serve as proxies for voter blocs; platform sentiment is then correlated against 14-day lagged favorability polls to derive elasticity estimates that supplement and cross-validate the news-derived signal. The fine-tuned LLM learns from both sources. The objective is to construct a stochastic, transparent, and replication-ready empirical framework that maps directly onto the model design described in the proposal. The guiding engineering principles are:

- **Reproducibility first.** All stochastic elements derive from a single global random seed and produce identical outputs across runs. The seed contract is explicit and non-negotiable:
  - `rng.py` provides `make_rng(seed)` returning a seeded `np.random.Generator` and `derive_seed(base, stage_name)` for per-stage sub-seeds.
  - Every stage function in `stages.py` must accept a `seed: int` argument and pass it explicitly to all downstream stochastic components.
  - Sklearn components that accept `random_state` (SetFit trainer, any cross-validation) must be called with `random_state=derive_seed(seed, "setfit")` — never relying on the global `np.random` state.
  - CVXPY solvers are deterministic given the same problem; no seed needed.
  - Dirichlet sampling uses `rng.dirichlet(alpha, size=N)` from the seeded generator, not `np.random.dirichlet`.
  - `test_rng.py` must include a test that runs the full pipeline twice with the same seed and asserts artifact equality field-by-field.
- **Modular architecture.** Economic kernels (voter distribution, sentiment elasticity, optimization) are strictly separated from orchestration logic.
- **Typed artifacts.** Every pipeline stage writes a structured artifact with an explicit schema. Tabular stages (panel, scored posts, fine-tuning dataset) write Parquet for the data payload; non-tabular stages (optimizer, simulation) write JSON. All envelopes (stage, run_key, metadata) are JSON.
- **Validation as infrastructure.** Each stage enforces data invariants and raises informative errors when they are violated.
- **Correctness over speed.** Code clarity and mathematical fidelity dominate premature optimization; HPC resources are reserved for LLM fine-tuning (Week 5) and Monte Carlo simulation (Week 6).

Compute Resource Allocation

Six compute environments are available. The guiding principle is **no machine sits idle**: while the M5 runs interactive development, the HPC runs a batch job, the Intel Mac runs the news scraper and nightly pipeline, and the Pi classifies bios. Shared storage (Syncthing) ties the three local machines (M5, Intel Mac, Pi) together; the HPC is staged separately via `rsync`/`scp`. The Windows laptop is not currently available.

CONCEPT: SLURM and HPC array jobs

SLURM is the job scheduler used on CMC's HPC cluster. You submit a shell script with `#SBATCH` directives (CPUs, GPUs, memory, time limit) using `sbatch`. SLURM queues the job, allocates a node when one becomes available, runs the script, and releases the node. A SLURM **array job** (`#SBATCH --array=0-24`) runs the same script 25 times in parallel, each with a different `$SLURM_ARRAY_TASK_ID` environment variable. Instead of scoring 25 shock events serially (2–3 days), an array job scores all 25 simultaneously, finishing in under 2 hours.

WATCH OUT

Never install Syncthing on the HPC. HPC compute nodes are ephemeral — allocated for a job, then reclaimed. Background daemons like Syncthing are killed when the node is reclaimed, and all sync state is lost. Jobs that depend on Syncthing completing a transfer before they start will fail silently. HPC data transfer uses explicit `rsync` commands *before* submitting each SLURM job. This is more manual but guaranteed reliable.

Machine	Primary Role	Secondary Role
M5 MacBook Pro (48GB)	Interactive dev, MLX fine-tune experiments, Claude sessions	HPC job monitoring; architecture sessions
Intel MacBook (2019)	Social media collection 24/7 (Bluesky firehose + Apify per-shock)	Nightly continuous pipeline (launchd, 2am); FastAPI endpoint during Week 7 testing
Intel MacBook (2019)	News scraper (was Windows role), Bluesky/Apify collection, nightly pipeline	launchd 2am; news scraper + collectors
Raspberry Pi 5 + AI HAT	Bio classification inference on NPU (always-on)	OpenClaw/Gemini gateway (CPU)
HPC (CMC)	LLM fine-tuning, RoBERTa SLURM array jobs, Monte Carlo	vLLM model serving for web app backend
Claude Pro Max	Architecture, debugging, LaTeX, complex reasoning	—
Gemini Pro	Boilerplate generation, bulk tasks, long-context ingestion	—

Shared storage (four local machines only): Install Syncthing on the M5, Intel Mac, and Raspberry Pi. **Do not install Syncthing on the HPC.** HPC cluster nodes enforce policies that kill background daemons; a Syncthing daemon on a login or scratch node will be terminated by automated scripts, silently starving any SLURM jobs that depend on it for data. Files written by the Intel Mac (collected posts) are immediately readable by the Pi (bio classifier), and M5 (development) via Syncthing. Intel Mac also on Syncthing for news articles. For the HPC, all data is staged explicitly using `rsync` or `scp` before job submission (see Week 4 Day 5 for the staging procedure).

Write-once convention (critical): Syncthing handles large static files well but will produce sync conflicts if multiple machines write to the same file concurrently. Enforce a strict ownership rule: each file in `rawdata/` is written by exactly one machine and never overwritten by another. Naming convention: `rawdata/social/{platform}/{shock_id}/{machine_id}_posts.jsonl`. The Intel Mac owns all `social/` writes; Intel Mac owns all `articles/` writes; neither touches the other's directory. For high-concurrency write sinks (embedding cache, audit log) use DuckDB on the M5 as the single writer; other machines append via HTTP to the M5's DuckDB endpoint, not directly to the Parquet files.

Merge stage (required before scorer). Because multiple files may exist per shock (one per platform per collector run), the Prefect DAG must include a `merge_posts(shock_id)` task before the RoBERTa scorer runs. This task finds all `rawdata/social/{platform}/{shock_id}/.jsonl` files across all platforms, concatenates them into a single `rawdata/merged/{shock_id}/posts.jsonl`, and logs the total post count per platform. The scorer always reads from `rawdata/merged/`. This makes multi-platform collection transparent to all downstream stages — no stage needs to know how many files or platforms contributed.

Raspberry Pi 5 + AI HAT+ (Hailo-8L, 26 TOPS): The NPU runs `all-MiniLM-L6-v2` for bio embedding inference after the model is compiled to Hailo Executable Format (HEF) using the Hailo Dataflow Compiler. Sub-millisecond per-bio latency on the NPU. The Pi's CPU simultaneously runs the OpenClaw gateway. The NPU and CPU operate independently with no resource competition.

HPC open-source model serving: Use `vLLM` to serve Mistral 7B on an A100 node with an OpenAI-compatible API endpoint. At 2,000 tokens/second on an A100, the entire fine-tuning dataset can be run through the base model for baseline comparison in minutes. SLURM array jobs parallelize RoBERTa scoring across 25+ shock events simultaneously.

Deployment Strategy

The web application is deployed to **Modal** from Day 1 of Week 8. Modal is the primary public deployment throughout the SRP period. An **HPC-hosted vLLM** alternative is scaffolded in Week 7 as a drop-in replacement, ready to activate once CMC HPC external connectivity is confirmed. Switching between Modal and HPC requires changing a single environment variable (`INFERENCE_BACKEND`).

Backend	When	Status
Modal (serverless GPU)	Weeks 7–8 and beyond	Primary — deploy immediately
HPC vLLM endpoint	Post-SRP (when familiar)	Scaffolded — one env var to activate
M5 local + ngrok	SRP poster demo only	For demo day, not public

Archive Storage Strategy

The combined raw archive corpus (Kavanaugh 56M tweets, 3DLNews2 8M+ HTML pages, Webhose, social media datasets) runs to several hundred gigabytes. Storing everything locally on every machine is unnecessary and impractical. The guiding principle is **fetch, extract, discard**: raw archives live only on the machine doing the processing; only the cleaned, sampled outputs flow through Syncthing to other machines.

Storage allocation:

Machine	What it stores	Size
Intel Mac	Raw social media archives (Kavanaugh, 2020 election, etc.), 3DLNews2 URL list, Webhose dumps, sampled outputs (`rawdata/social/` and `rawdata/articles/`), cleaned JSONL files ready for scoring	Primary archive machine; has available local disk
HPC `$SCRATCH`	Raw 3DLNews2 HTML pages during fetch-and-parse; merged post files during SLURM scoring jobs. Scratch is ephemeral — wipe after each job completes	Terabytes available; use aggressively
M5	Processed outputs only (embeddings cache, fine-tuning dataset, artifacts). No raw archives.	Small footprint
Syncthing	Syncs **processed outputs only**: sampled JSONL, cleaned JSONL, scored embeddings. Never syncs raw archives.	Stays small

3DLNews2 fetch-and-parse workflow. Never store 8M raw HTML pages locally. Instead:

1. Download the URL metadata file to the Intel Mac (`data/archives/news/3dlnews/`) — this is small (URLs + dates + outlet names, a few GB).
2. Filter the URL list to your shock date windows and target states *before* fetching any HTML. This reduces the fetch set to tens of thousands of URLs, not millions.
3. Run the fetch-and-parse on HPC scratch via a SLURM array job: each node fetches a batch of URLs, strips HTML to plain text, applies the 100-word minimum, and writes `articles.jsonl` directly. The raw HTML is never written to disk.
4. `rsync` the resulting `articles.jsonl` files back to the Intel Mac. Discard nothing on HPC scratch — it will be wiped automatically.

Social media archives. The Kavanaugh dataset (56M tweets, ∼80GB compressed) and similar large archives download directly to the Intel Mac. Run `sample_archives.py` on HPC scratch (stage via `rsync`, run, retrieve the 200k-post sample). Keep only the sampled JSONL on the Intel Mac; the full raw archive can be deleted once sampling is confirmed correct. The sample is ∼200MB — a negligible fraction of the raw size.

Efficiency and Quality Upgrades

The following improvements are incorporated into the pipeline design and scheduled across the 7-week plan. Each addresses a specific bottleneck or quality gap.

A — Parquet + DuckDB artifact store (Week 1). All tabular pipeline stages (voter panel, scored posts, fine-tuning dataset) write Parquet rather than JSON for the `data` payload. The artifact *envelope* (stage, run_key, metadata) stays JSON. DuckDB queries Parquet directly without loading into memory. Result: 5–10× faster reads, smaller Syncthing footprint, SQL-queryable corpus for the analyst dashboard.

B — SetFit bio classifier (Week 4). Replace k-NN over sentence embeddings with a SetFit classification head trained via contrastive learning on the 100–200 labelled bios per bloc. SetFit reaches strong accuracy with as few as 8 examples per class, generalizes better to novel phrasing,

and is still small enough to compile to Hailo HEF for the Pi NPU. Better bio accuracy → more precise social media signal → better fine-tuning data.

C — Streaming SSE inference (Week 7). Replace blocking HTTP with server-sent events (SSE) so partial results stream as each pipeline stage completes: LLM deltas appear first (fast), then optimizer weights, then win probability. FastAPI native SSE; React `EventSource`. Half a day of implementation, dramatically more responsive UX for the demo and the public app.

D — Synthetic training data via Gemini (Week 4). Use Gemini 2.5 Pro's 1M-token context window to ingest the full voter panel, all 25+ shock records, and the dev plan, then generate 500–1,000 synthetic training examples. Labeled as synthetic and weighted at 0.5 in the regression.

Mode collapse prevention (critical): Fine-tuning Mistral 7B heavily on LLM-generated data risks the model overfitting to Gemini's sociopolitical priors rather than learning empirical electoral equilibrium.

Do not hard-reject batches by MMD threshold. A hard MMD gate that discards all batches where $\text{MMD}^2 > \tau$ will filter out precisely the synthetic examples where historically-correlated blocs decouple — e.g., White Evangelicals and White Catholics moving in opposite directions due to a theological-political crisis. These novel co-movements are exactly what the model needs to learn in order to generalize beyond historical patterns. A hard gate converts the model into a mechanism for projecting the past forward, defeating the stated purpose of dynamic shock forecasting.

Use a soft MMD-weighted penalty instead. Rather than rejecting batches, weight each synthetic example in the regression by:

$$w_{\text{synth}} = 0.5 \times \exp\big(-\lambda \cdot \text{MMD}^2(P_{\text{synth}}, P_{\text{real}})\big)$$

where λ is a **fixed theoretical constant, not bootstrap-estimated**. With $N < 30$ historical events, bootstrapping λ from the real data in an infinite-dimensional RKHS will produce extreme variance — there is not enough data to map the true manifold of P_{real}, making bootstrap-derived λ statistically unstable.

Instead, set λ analytically from the RBF kernel bandwidth σ: given an RBF kernel $k(x, y) = \exp(-\|x - y\|^2/2\sigma^2)$, the median heuristic sets $\sigma^2 = \text{median}(\|x_i - x_j\|^2)$ over the real data pairs. Then set $\lambda = 1/(2\sigma^2)$ (the inverse kernel bandwidth), which ensures that two distributions at the kernel's characteristic scale receive weight $\sim e^{-1} \approx 0.37$ and perfectly matching distributions receive weight 1.0. This is deterministic, interpretable, and does not require bootstrap estimation. Store in `configs/mmd_config.json` as `"lambda": <computed value>` and log the median pairwise distance used to compute it. Implement in `scripts/generate_synthetic.py`: compute per-batch MMD, assign the weight above to each example, write `mmd_weight` to the output JSONL. The training loop multiplies each example's loss by `mmd_weight * 0.5` (the base synthetic weight).

Additional diagnostics required alongside MMD (write to `data/finetune/synthetic_diagnostics.json`):

- **PCA alignment.** Fit PCA on the real historical $\Delta\mu$ matrix (rows = events, columns = stratum cells). Project the synthetic batch onto the same PCA basis. Compute the fraction of synthetic variance explained by the top-2 real principal components. If <70%,

the synthetic data is occupying a different subspace than the historical data — flag this batch for manual inspection.

- **Pairwise Correlation Difference (PCD).** Compute the full pairwise correlation matrix of $\Delta\mu$ cells for the real data (C_{real}) and the synthetic batch (C_{synth}). Report PCD $= \|C_{\text{real}} - C_{\text{synth}}\|_F / K(K-1)$ (Frobenius norm normalized by the number of off-diagonal elements). Low PCD confirms the synthetic data respects historical covariance structure; high PCD means the batch is generating demographically impossible co-movement patterns.

Report PCA alignment and PCD in the paper's synthetic data validation section alongside MMD. Three independent metrics are more convincing to reviewers than one.

E — Gaussian Process baseline model (Week 3). Replace XGBoost with a Gaussian Process classifier using an RBF + Matern kernel. GPs model temporal correlations explicitly (2020 is more informative about 2024 than 2000 is), provide calibrated uncertainty estimates out of the box, and scale fine with only 6–8 cycles. Swap: `GaussianProcessClassifier` from `sklearn.gaussian_process`. The uncertainty estimates feed directly into the paper's epistemic humility framing.

G — Prefect DAG orchestrator (Week 1). Replace hand-written `pipeline.py` with Prefect flows. Benefits: automatic retry on failure, parallel stage execution where stages are independent, a visual dashboard showing stage completion status, and persistent state so a mid-pipeline crash restarts from the last successful stage rather than the beginning. Runs locally with zero infrastructure; free cloud dashboard.

H — Cache RoBERTa embeddings, not scores (Week 4). Compute and cache raw RoBERTa embedding vectors per post to disk (Parquet). Sentiment scoring and aggregation run on top of cached embeddings. Embeddings are deterministic and expensive; aggregation is cheap and will be iterated on. Re-running the elasticity regression after a parameter change goes from hours to seconds.

Palantir-aligned additions (Week 7). Two additions specifically designed to align the project with Palantir's FDE profile: (i) an **analyst dashboard** (password-protected, separate from the public app) showing pipeline internals — data coverage by bloc and cycle, RoBERTa score distributions, bio classifier coverage rates, fine-tuning loss curves, Monte Carlo convergence diagnostics; (ii) **audit logging** — every `/estimate` call persists input, deltas, optimizer feasibility, and win probability to a DuckDB database with a query interface. These demonstrate the difference between a tool for end users and a tool for analysts — Palantir's entire business model.

I — Async FastAPI inference pipeline (Week 7). The `/estimate/stream` SSE endpoint streams three stages in sequence. Each stage must be offloaded correctly for its concurrency model:

- **CVXPY optimizer** — use `ProcessPoolExecutor` (not `ThreadPoolExecutor` and not `None`). CVXPY's C solvers are not thread-safe; passing `None` defaults to a thread pool and will cause segfaults under concurrent requests.
- **Monte Carlo** (pure NumPy) — safe in a `ThreadPoolExecutor`; NumPy releases the GIL during array operations.

Expected latency reduction: ∼30% on total round-trip by streaming each stage result to the client while the next stage begins.

J — Batch bio classification (Week 4). Social media collectors currently call `http://$PI_TAILSCALE_IP:9000/classify` once per post inline during collection. At 1,000 posts × 5ms round-trip = 5 seconds of serial HTTP overhead per shock event. Add `POST /classify_batch` to `pi_bio_server.py` accepting a list of `(bio, lang)` pairs. Batch posts into groups of 50 before calling. The Hailo NPU processes a batch as fast as a single item; HTTP overhead drops by 50×.

K — Justfile for daily commands (Week 1). Install `just` (`brew install just`). Write a `Justfile` at the repo root with targets: `smoke` (runs pipeline on smoke config), `test` (runs pytest), `score` (submits RoBERTa SLURM array), `train` (submits HPC fine-tuning job), `deploy` (runs modal deploy), `sample` (submits stratified archive sampling SLURM array), `clean` (runs Gemini Pro LLM cleaning on sampled data), `continuous` (runs nightly incremental pipeline post-SRP). Saves 7 weeks of typing long commands and eliminates flag typos.

Cost Strategy: Spend Only Where It Matters

This project is student-funded. Every paid service has a free tier, a student programme, or a CMC-provided alternative. Default to free; pay only when a paid tier directly unblocks research progress.

Service	Cost	Programme	Notes
CMC HPC	Free	CMC student allocation	Fine-tuning, scoring, Monte Carlo
Modal (GPU inference)	∼$0.0003/call	$30/mo free credit	100k calls free/month
Vercel (frontend)	Free	Hobby tier	Next.js hosting, unlimited
GitHub	Free	Student Developer Pack	Private repos, Codespaces
Namecheap domain	Free	Student Pack	via GitHub Student Pack
Gemini Pro	Free	Google AI Studio API	60 RPM, 1k req/day free
Bluesky AT Protocol	Free	Open source	Primary live collection; full firehose
Apify X Scraper	Free	Free tier	Secondary live collection; 500 results/run
Syncthing	Free	Open source	Self-hosted shared storage
Prefect	Free	Open source + free cloud	DAG dashboard, no infra needed
Claude Pro Max	Already paid	Existing subscription	Reserve for hard problems

Total expected cost: $0–$5/month during development. Modal GPU spend depends on demo traffic.

Apply immediately (Week 0):

1. **GitHub Student Developer Pack** at `education.github.com/pack` — unlocks JetBrains IDEs, Namecheap domain, Canva Pro, and more.

2. **Social media access — entirely free.** Two live collection paths and one archive path, all at no cost: (a) **Bluesky** (primary live) — sign up at `bsky.app`, install `atproto`. Full firehose, no rate limits, no cost. (b) **Apify X Scraper** (secondary live) — sign up at `apify.com`.

Free tier gives 500 results/run. Covers X demographic signal without any API subscription. (c) **Historical Twitter archives** (primary volume) — 56M Kavanaugh tweets (Pushshift), 3.5M 2020 election tweets (IEEE DataPort), 2020 Kaggle election dataset, MeToo tweets (Data.world) and others. These are free downloads covering the most important shock events directly. Do not sign up for the X Basic API ($100/month) — the archive coverage is superior for historical events and Bluesky/Apify covers live collection.

3. **Meta Content Library** at `developers.facebook.com/docs/content-library` — free academic access. Approval takes 2–4 weeks.

4. **Modal account** at `modal.com` — $30/month free credit. At Mistral 7B on an A100 (∼3 seconds per call), $30 covers ∼100,000 inference calls. No warm-container overhead during development; use cold starts and save the credits for the demo.

Technical Review Responses

The following structural risks were identified in external review and addressed:

Syncthing concurrent writes. Addressed by a strict write-once ownership convention: each file is written by exactly one machine. High-concurrency sinks (embedding cache, audit log) route through a DuckDB endpoint on the M5 rather than direct Parquet writes from multiple machines.

X API and XCollector removed. The X Basic API ($100/month) and all associated `XCollector`/`tweepy` code are not used. Historical Twitter data comes from free public archives (Kavanaugh 56M, 2020 election datasets, MeToo, etc.). Live collection uses Bluesky (free, AT Protocol) and Apify (free tier, X demographic profile). No X API calls are made at any point in the pipeline.

Static demographic bounds. Constraint bounds are now cycle-parameterized via `build_constraint_spec(panel_df)`, derived from actual eligible voter pool fractions rather than hardcoded constants.

CVXPY thread safety. The async SSE endpoint now uses `ProcessPoolExecutor` (not `ThreadPoolExecutor`) for CVXPY calls, with a `solver_args={"threads": 1}` safety flag. Monte Carlo (pure NumPy) remains in a thread pool.

Synthetic data mode collapse — MMD gating. Per-bloc univariate KS tests were insufficient: a batch can pass all individual KS gates while presenting demographically impossible joint comovements. Replaced with Maximum Mean Discrepancy (MMD) on the full joint $\Delta\mu$ vector, using an RBF kernel with bandwidth set by the median heuristic over the real historical data. The scaling constant λ is set analytically from the kernel bandwidth rather than bootstrap-estimated (bootstrap is statistically unstable at $N < 30$ events in an infinite-dimensional RKHS). Synthetic examples receive a soft weight $w = 0.5 \times \exp(-\lambda \cdot \text{MMD}^2)$ rather than a hard rejection threshold.

LLM float estimation. Replaced direct float output with 9-token categorical bins. A `bin_to_delta` function maps tokens to numeric midpoints; a `bin_to_distribution` function returns the bin range for Monte Carlo uncertainty propagation.

Bio classifier language independence violation. The language signal is now a fallback-only path (no bio signal found) rather than a multiplicative factor. Combining the two assumed statistical independence between bio content and tweet language, which is false.

Dirichlet replaced by Logistic-Normal Monte Carlo with ILR parameterization. The Dirichlet's off-diagonal covariances are strictly negative, making it impossible to model positively correlated demographic shifts. Replaced with the Logistic-Normal distribution parameterized via isometric log-ratio (ILR) coordinates using a Helmert contrast matrix, rather than the delta method approximation ($\Sigma_{\ln,ij} \approx \Sigma_{\Delta,ij}/(w_i^* w_j^*)$). The delta method floor ($w_i^* \geq 0.01$) introduced severe non-linear distortion near simplex boundaries in Aitchison geometry — a 0.01 perturbation in probability space is not 0.01 in log-ratio space. The ILR approach avoids this by working in orthonormal coordinates that are better conditioned throughout the interior of the simplex. Blocs assigned $w_i^* = 0$ by the optimizer are reported as `infeasible_bloc` rather than floored.

Schema redundancy (`BaselinePortfolioData`). Not a bug: both fields are in a `frozen=True` dataclass, making mutation impossible after construction. State mismatch between `weights` and `mu` cannot occur at runtime.

Syncthing on HPC SLURM array nodes. The Week 4 SLURM array job previously loaded `posts.jsonl` from Syncthing. HPC scratch nodes kill background daemons silently. Fixed: `rsync` transfers all social and articles data to `$SCRATCH` before `sbatch` is submitted. The array job reads from `$SCRATCH` only.

Prediction market dynamic slug resolution. Querying Polymarket, Manifold, and Metaculus with generic slugs returns empty or irrelevant results — each platform uses its own internal contract IDs. Fixed by adding `configs/market_contracts.json`: a static, manually-built mapping from every shock event to its exact contract identifier per platform.

Manual `json_schema()` vs Pydantic. The original implementation built a raw JSON Schema dict by hand. `outlines` natively accepts Pydantic models; the `ShockResponseSchema` Pydantic model (using `Literal` for the 9 bin tokens) is now passed directly. This guarantees the LLM output schema and the FastAPI `response_model` are always the same Python object and can never drift out of sync.

SSE executor: `None` defaults to thread pool. Passing `None` to `run_in_executor` uses `ThreadPoolExecutor`. CVXPY's C solvers are not thread-safe. Fixed: CVXPY runs in a `ProcessPoolExecutor(max_workers=1)` instantiated at startup. Monte Carlo (pure NumPy, GIL-releasing) stays in a thread pool.

SetFit CPU fallback crashes on Hailo initialisation. When `pi_npu_enabled=false`, the bio server previously attempted to pass data to a non-existent Hailo interface. Fixed: `pi_bio_server.py` reads `pi_npu_enabled` at startup and branches: NPU path loads HEF via Hailo SDK; CPU path loads `all-MiniLM-L6-v2` via HuggingFace `SentenceTransformer` into CPU memory. The Hailo SDK is never initialised in CPU mode. The `/health` endpoint reports the active mode.

HPC removed from Syncthing network. The architecture description previously said "Install Syncthing on all five physical machines." HPC cluster nodes kill background daemons; the daemon would be terminated by automated scripts, silently starving SLURM jobs that depend on it for data. Fixed: Syncthing is installed on the three local machines only (M5, Intel Mac, Pi). All HPC data transfer uses explicit `rsync`/`scp` staging before job submission.

Dirichlet per-bloc α_i formula is mathematically correct. One reviewer suggested replacing the per-bloc formula with a single global precision scalar α_0 distributed as $\alpha_i = \alpha_0 \cdot w_i^*$. This is a different and less informative approximation. In a Dirichlet distribution, the concentration parameters α_i are specified independently; $\alpha_0 = \sum_i \alpha_i$ is a derived quantity, not an input constraint.

Computing each α_i from its own σ_i^2 correctly preserves the per-bloc uncertainty structure estimated by Ledoit-Wolf. The reviewer's alternative discards per-bloc variance information by treating all blocs as having equal relative uncertainty. Formula unchanged.

SetFit hardware boundary corrected. The bio inference section previously said "the trained SetFit model is compiled to HEF." Only the `all-MiniLM-L6-v2` embedding backbone is an ONNX transformer graph compilable to HEF. The SetFit logistic regression head is a scikit-learn classifier and cannot be compiled to HEF. Corrected: embedding backbone runs on NPU; logistic head runs on Pi ARM CPU using NPU-generated embeddings as input features.

HPC scratch volatility: auto-copy adapter weights. HPC scratch is ephemeral — once a compute node is reclaimed, `$SCRATCH` is wiped. Relying on a manual `scp` after the job exits risks losing adapter weights permanently. The final lines of `hpc_submit.sh` now automatically copy the adapter folder to `$HOME/projects/` (persistent across jobs) before the job exits.

Dirichlet near-degenerate test case added. The ≥ 0.01 floor on α_i was already present. Added a required `test_montecarlo_degenerate` test: $w_0^* = 0.99$, $w_{1..9}^* = 0.001$ — asserts no exception, all samples sum to 1.0, and `win_probability` is in $[0, 1]$.

Merge stage added before scorer. The write-once convention produces one `{machine_id}_posts.jsonl` per platform per shock. A new `merge_posts(shock_id)` Prefect task concatenates all per-platform files into `rawdata/merged/{shock_id}/posts.jsonl` before the scorer runs. The scorer always reads from `rawdata/merged/`; no stage knows how many files contributed.

Seed propagation contract made explicit. The reproducibility principle now specifies the full contract: every stage function accepts `seed: int`; sklearn components use `random_state=derive_seed(seed, "component_name")`; Dirichlet sampling uses the seeded generator; `test_rng.py` asserts artifact equality across two runs with the same seed.

DuckDB write concurrency: asyncio.Queue pattern. DuckDB is embedded; concurrent async coroutines writing to the same `.duckdb` file produce file lock errors. Fixed: `AuditLogger` uses an `asyncio.Queue` with a single background `Task` as the sole writer. Request handlers push entries non-blockingly; read queries use `read_only=True` mode.

Efficiency Principles

- **Vectorize everything.** NumPy batch operations: Monte Carlo as $(N \times 10)$ matrices, simplex projection row-wise, RoBERTa in batches of 32.
- **Never recompute.** Artifacts cached to disk; idempotent stages skip if output exists. RoBERTa embeddings cached separately from scores (Option H).
- **Route by cost.** Gemini: boilerplate, bulk generation, long context. Claude: architecture, debugging, reasoning. HPC: batch. Pi NPU: always-on.
- **Parallelize across machines.** Weeks 4–5: Intel Mac (collection), Pi (bio classification), Intel Mac (scraping + collection), HPC (scoring), M5 (development).
- **Reproducibility first.** Single global seed. Prefect DAG for restart-from-failure. Parquet artifacts for deterministic I/O.

Juhi — Primary Objective. Design, implement, test, and deploy the full research pipeline independently, from historical data ingestion through LLM fine-tuning, stochastic optimization, and web application deployment. AI tools (Claude via claude.ai, Gemini via OpenClaw) serve as the primary development partner throughout — for code generation, debugging, LaTeX writing, and architectural decisions.

Prof. Espinosa — Supervisory Role. Provide domain expertise on the 10 key voter blocs, validate demographic conventions and equilibrium threshold assumptions, and offer high-level guidance on the research direction. Prof. Espinosa does not implement any code or pipeline components. His role is to review outputs, answer domain questions, and supervise the intellectual framing of the project.

Execution Model

Development will proceed in structured weekly increments. The execution model follows four principles:

- **Stage-based development.** Each week delivers one primary pipeline stage, implemented end-to-end (kernel, validation, tests) before moving forward.
- **Pull-request discipline.** All new functionality is submitted via pull request with unit tests and documentation. No stage is considered complete without passing tests.
- **Seed conventions.** Every stochastic component derives randomness from the global run seed using documented sub-seed conventions.
- **Incremental integration.** Stage kernels are developed independently but integrated into the pipeline immediately to prevent architectural drift.

The weekly cadence is:

1. *Early week:* Juhi implements kernel logic and associated unit tests, using Claude and Gemini as development partners.
2. *Mid-week:* Pull request submitted with documentation and validation checks.
3. *End of week:* Juhi merges after tests pass; brief check-in with Prof. Espinosa for domain feedback and supervision.

2 High-Level Development Snapshot (7-Week Build Plan)

Week	Stage	Outcome
Week 0 (Planning)	Devplan Review & Environment Setup	Devplan finalised, methodology confirmed with Prof. Espinosa, all infrastructure running (Tailscale, Syncthing, Hailo HEF, collectors, Meta Content Library applied).

Week	Stage	Outcome
Week 1	Historical Data Pipeline & Voter Panel	Cleaned longitudinal voter panel from ARDA, GSS, Gallup, NEP, and Pew; artifact schemas, seed conventions, and validated JSON I/O for all stages.
Week 2	Baseline Voter Portfolio	ML-derived baseline demographic distribution; V_{eq} computed; CVXPY baseline optimizer validated.
Week 3	RoBERTa Pipeline, Social Media & Fine-Tuning Dataset	News corpus scored; Bluesky, Apify, Reddit, Facebook, Truth Social, Telegram sentiment collected; unified fine-tuning dataset assembled.
Week 4	LLM Fine-Tuning & Inference Endpoint	Mistral 7B fine-tuned via QLoRA on HPC; FastAPI endpoint with constrained decoding; four-way baseline comparison (including historical mean).
Week 5	Stochastic Optimization & Monte Carlo	CVXPY max-P(win) optimizer; Logistic-Normal Monte Carlo with 90% CI band; SSE streaming.
Week 6	Web Application	Next.js frontend; CoalitionChart with CI band; WinGauge with shaded range; ShockNarrative; analyst dashboard live.
Week 7	Hardening, Validation & Deliverables	Integration tests, determinism gates, replication package, research paper draft, and technical poster finalized.

3 Week 0 (Planning): Devplan Review, Infrastructure, and Environment

Objective of Week 0 (Planning)

Week 0 is not a build week. Nothing is coded. The goal is to arrive at Week 1 with complete methodological clarity so no time is lost mid-build re-architecting decisions that should have been made before writing a line of code.

By the end of Week 0, the following must be true:

- Every section of the devplan has been read and confirmed with Prof. Espinosa
- Voter panel sources (ARDA, GSS, NEP) confirmed accessible and confirmed to provide race × religion × gender cross-tabulations
- V_{eq} computation methodology agreed
- Tailscale installed on all five machines and Pi resolves at a stable IP
- Syncthing running on four local machines; HPC rsync tested
- Hailo HEF compilation attempted; CPU fallback confirmed working if it fails

- Bluesky and Apify collectors running and logging to Syncthing
- Meta Content Library application submitted
- Reddit API credentials obtained (OAuth, 100 req/min free tier)
- Open-weight cleaning model selected (`Qwen2.5-7B-Instruct` or `Mistral-7B-Instruct-v0.3`) and tested on 10 sample posts
- `DECISIONS.md` created with initial architectural decisions

> *Given a configuration file and a global seed, the pipeline produces one typed artifact per stage with a stable envelope schema, and each artifact can be parsed into a validated Python object. Tabular stages write Parquet; all others write JSON.*

No economic or statistical functionality is implemented this week. Week 1 defines the structural contract that all future stage logic must obey.

CONCEPT: Artifact-first design

Each stage of the pipeline produces a frozen Python dataclass (an **artifact**) that the next stage consumes as its only input. The discipline: no stage reads from a file path, database, or global variable that was not explicitly passed through an artifact. This makes stages independently testable — you can re-run any stage by providing its input artifact. It makes failures visible: "VoterPanelData is missing `cell_share`" appears immediately rather than producing silent corruption three stages later.

CONCEPT: Prefect

Prefect is a Python workflow orchestration library. You decorate functions with `@task` and compose them into a `@flow`. Prefect handles retry logic (if the HPC SLURM submission times out, retry 3 times before alerting), structured logging (every task run is searchable), and dependency tracking (Task B only runs after Task A completes successfully). Without orchestration, a failing stage silently corrupts downstream data. With it, failures surface immediately and the pipeline is self-documenting.

WHY THIS DESIGN CHOICE

Why define artifact schemas before writing any pipeline logic? Because schema changes are the most expensive kind of refactor. If Week 4 discovers that the scorer needs a field that VoterPanelData does not provide, every stage between Week 2 and Week 4 needs to be rewritten. Defining the schema first forces those discoveries in Week 1, when the cost of fixing them is zero.

Artifact Envelope Contract (Applies to All Stages)

All stage artifact files must conform to the following schema:

```
{
  "stage":    "...",
  "run_key":  "...",
```

```
  "metadata": {...},
  "data":     {...}
}
```

The `data` field must be parseable into a typed stage payload class.

Week 1 Responsibilities

Juhi implements all code, tests, and infrastructure this week. Prof. Espinosa's only input is a one-time domain confirmation: the canonical bloc names, the equilibrium thresholds ($V_{\text{eq}}^{D} \approx 0.52$–$0.53$ for Democrats, $V_{\text{eq}}^{R} \approx 0.49$–$0.51$ for Republicans, derived empirically from the voter panel — see Week 3), and the `party` field conventions, recorded in `DECISIONS.md`.

- `electoral/artifacts.py` (expand with payload classes)
- `electoral/core/schema.py` (create)
- `electoral/core/types.py` (create)
- `electoral/core/io.py` (Parquet + JSON artifact I/O)
- `electoral/stages.py` (create with one stub per stage)
- `electoral/pipeline.py` (Prefect flow)
- `Justfile` (shortcut commands)
- `tests/test_artifact_roundtrip.py` (create)

Part 1: Stage Payload Data Classes

Each payload class must be an immutable dataclass implementing:

```
def to_dict(self) -> dict[str, Any]: ...
@classmethod
def from_dict(cls, payload: dict[str, Any]) -> "ClassName": ...
def validate(self) -> None: ...
```

Design rule: JSON-first, minimal but explicit. Stage payloads must use only native Python types (`dict`, `list`, `str`, `int`, `float`, `bool`, `None`). Do not store pandas or NumPy objects in payload fields. All election cycle keys use canonical `YYYY` integer format. All demographic identifiers must be strings.

The required payload classes for Week 1 are summarised below.

VoterPanelData — cleaned longitudinal panel.

```
@dataclass(frozen=True)
class VoterPanelData:
    cycles:      list[int]   # election years, sorted unique (YYYY)
    races:       list[str]   # ["african_american","latino","asian","white
       ","other_race"]
    religions:   list[str]   # ["evangelical","catholic","protestant",
```

```
                                #  "secular","jewish","muslim","other_rel"]
    genders:     list[str]    # ["women","men","other_gender"]
    # Three independent marginal tables; no cross-tabulations
    n_rows_race:     int      # rows in panel_race.parquet
    n_rows_religion: int      # rows in panel_religion.parquet
    n_rows_gender:   int      # rows in panel_gender.parquet
    layer_weights:   dict[str, float]
    # {"lambda_1": 0.xx, "lambda_2": 0.xx, "lambda_3": 0.xx}; sum to 1
    source:      str | None   # e.g. "ARDA+GSS+NEP"
```

BaselinePortfolioData — steady-state voter equilibrium.

```
@dataclass(frozen=True)
class BaselinePortfolioData:
    method:      str
    party:       str            # "democrat" or "republican"
    weights:     dict[str, float]
    # weights[race_id] = optimal coalition weight (5 keys); must sum to
       1.0
    mu_race:         dict[str, float]
    # mu_race[race_id] = within-race Democratic vote share (Stratum 1)
    mu_religion:     dict[str, float]
    # mu_religion[religion_id] = within-religion Democratic vote share (
       Stratum 2)
    mu_gender:       dict[str, float]
    # mu_gender[gender_id] = within-gender Democratic vote share (Stratum
       3)
    mu_eff:          float
    # mu_eff = lambda_1*(sum w_i*mu_race_i) + lambda_2*(sum v_R*mu_rel_R)
    #        + lambda_3*(sum g_G*mu_gen_G); scalar used by optimizer
    layer_weights:   dict[str, float]  # {"lambda_1":..,"lambda_2":..,"
       lambda_3":..}
    target:          float
    # equilibrium threshold: ~0.52-0.53 for Dem, ~0.49-0.51 for Rep (from
       voter panel)
```

SentimentData — RoBERTa-derived elasticity metrics (used as fine-tuning input features, not as final shock estimates).

```
@dataclass(frozen=True)
class SentimentData:
    model:    str            # e.g. "cardiffnlp/twitter-roberta-base-
       sentiment"
    shocks:   list[str]      # shock event identifiers
    scores:   dict[str, dict[str, float]]
    # scores[bloc_id][shock_id] = sentiment elasticity in [-1, 1]
    # NOTE: these scores are intermediate features, not final estimates.
    # They are consumed by the LLM fine-tuning pipeline in Week 5.
```

LLMFineTuneData — training dataset constructed from RoBERTa outputs paired with historical polling deltas.

```
@dataclass(frozen=True)
```

```
class LLMFineTuneData:
    base_model:   str          # e.g. "mistralai/Mistral-7B-v0.3"
    lora_rank:    int          # LoRA rank parameter (e.g. 16 or 32)
    n_examples:   int          # number of fine-tuning examples
    cycles_used:  list[int]    # election cycles in training set
    adapter_path: str | None   # path to saved LoRA adapter weights
```

SocialMediaSentimentData — platform-aggregated sentiment paired with lagged favorability polls, used as a high-frequency weak supervisor to augment the small election-cycle training set.

```
@dataclass(frozen=True)
class SocialMediaSentimentData:
    shock:        str              # shock event identifier
    platforms:    list[str]        # e.g. ["bluesky", "apify", "facebook",
        "truth_social", "telegram"]
    window_hours: int              # collection window around shock (e.g.
       72)
    scores:       dict[str, dict[str, float]]
    # scores[platform][bloc_proxy] = elasticity in [-1, 1]
    # bloc_proxy is platform-derived demographic proxy, not direct bloc
       measure
    n_posts:      dict[str, int]   # n_posts[platform] = number of posts
       collected
    lagged_delta: dict[str, float] | None
    # lagged_delta[bloc_id] = favorability shift at t+14 days (None if not
        yet available)
```

PredictionMarketData — incentive-compatible aggregate beliefs from real-money prediction markets, collected around each shock event.

```
@dataclass(frozen=True)
class PredictionMarketData:
    shock:          str              # shock event identifier
    party:          str              # "democrat" or "republican"
    # Pre-shock baseline: market-implied win probability 24h before shock
    pre_shock_prob: float            # in [0, 1]
    # Post-shock: market-implied win probability at 1h, 24h, 72h after
       shock
    post_shock_1h:  float | None
    post_shock_24h: float | None
    post_shock_72h: float | None
    # Implied belief update: the incentive-compatible signal
    delta_prob:     float            # post_shock_24h - pre_shock_prob
    sources:        list[str]        # e.g. ["polymarket", "predictit", "
       metaculus"]
    contract_ids:   dict[str, str]   # sources[i] -> contract identifier
    # volume[source] = total $ volume traded in the 72h window (data
       quality proxy)
    volume:         dict[str, float] | None
```

ShockResponseData — combined regression-derived voter share deviations, fusing news corpus, social media, and polling signals.

```
@dataclass(frozen=True)
class ShockResponseData:
    shock:            str
    cycle:            int
    party:            str           # "democrat" or "republican"
    # Delta bins per stratum --- three independent dictionaries
    delta_bins_race:     dict[str, str]
    # delta_bins_race[race_id] = one of 9 magnitude tokens (5 keys)
    delta_bins_religion: dict[str, str]
    # delta_bins_religion[religion_id] = one of 9 magnitude tokens (7 keys
       )
    delta_bins_gender:   dict[str, str]
    # delta_bins_gender[gender_id] = one of 9 magnitude tokens (3 keys:
       women/men/other)
    # For non-gendered shocks, women==men==other_gender bins
    # For non-religious shocks, all religion bins identical
    deltas_race:     dict[str, float]   # numeric midpoints of
       delta_bins_race
    deltas_religion: dict[str, float]   # numeric midpoints of
       delta_bins_religion
    deltas_gender:   dict[str, float]   # numeric midpoints of
       delta_bins_gender
    delta_eff:       float
    # Scalar effective delta = lambda_1*(sum w_i*delta_race_i)
    #                        + lambda_2*(sum v_R*delta_rel_R)
    #                        + lambda_3*(sum g_G*delta_gen_G)
    covariance:      list[list[float]]  # 5x5 race-level covariance of
       deltas_race
    source:          str
    # one of: "llm_unified", "roberta_news_only", "roberta_social_only"
```

EquilibriumData — rebalanced voter distribution after shock.

```
@dataclass(frozen=True)
class EquilibriumData:
    method:           str
    party:            str          # "democrat" or "republican"
    shock:            str | None
    weights:          dict[str, float]
    # weights[race_id] = rebalanced coalition weight; 5 keys; sums to 1.0
    mu_eff_shifted:   float
    # post-shock effective loyalty scalar (all strata combined)
    # = lambda_1*(sum w_i*(mu_race_i+delta_race_i))
    # + lambda_2*(sum v_R*(mu_rel_R+delta_rel_R))
    # + lambda_3*(sum g_G*(mu_gen_G+delta_gen_G))
    feasible:         bool
    target_met:       bool
    target:           float        # configurable per party
```

SimulationData — Monte Carlo simulation summary.

```
@dataclass(frozen=True)
class SimulationData:
    n_simulations:     int
```

```
    seed:              int
    win_probability:   float     # point estimate: fraction of runs
       meeting V_eq
    win_probability_low:  float  # 5th percentile (p=0.05 lower bound)
    win_probability_high: float  # 95th percentile (p=0.95 upper bound)
    # 90% CI band displayed as shaded range on WinGauge
    # "No feasible path" fires when win_probability_high < V_eq
    # "Uncertain path" fires when win_probability_low < V_eq <=
       win_probability_high
    percentiles:    dict[str, list[float]]
    # percentiles[bloc_id] = [p5, p25, p50, p75, p95] across simulations
```

MetricsTablesData — summary performance metrics.

```
@dataclass(frozen=True)
class MetricsTablesData:
    tables: dict[str, Any]
    # tables["table_a"] = JSON-serializable table payload
```

Part 2: Schema Helpers (`electoral/core/schema.py`)

```
def assert_required_keys(payload, keys, *, context="payload") -> None:
    """Raise ValueError if any required key is missing."""

def assert_unique(items, *, name="items", context="payload") -> None:
    """Raise ValueError if items contains duplicates."""

def assert_sorted_unique(items, *, name="items", context="payload") ->
   None:
    """Raise ValueError unless items is strictly increasing."""

def assert_valid_share(value, *, name, context) -> None:
    """Raise ValueError unless 0.0 <= value <= 1.0."""

def assert_shares_sum_to_one(d, *, context, tol=1e-6) -> None:
    """Raise ValueError if dict values do not sum to 1.0 within tol."""
```

Helpers must: raise `ValueError` with informative messages; be pure functions with no I/O or global state; accept a `context` string for error localisation.

Part 3: Type Aliases (`electoral/core/types.py`)

```
# electoral/core/types.py
from typing import TypeAlias, Literal

Cycle:            TypeAlias = int              # YYYY format (e.g. 2020)
BlocId:           TypeAlias = str              # Demographic identifier (e
   .g. "evangelical")
Weight:           TypeAlias = float            # Vote share in [0, 1]
ElasticityScore:  TypeAlias = float            # RoBERTa sentiment
   elasticity in [-1, 1]
ShockDelta:       TypeAlias = float            # LLM-estimated vote-share
   change
```

```
LoraRank:          TypeAlias = int               # QLoRA rank parameter
Platform:          TypeAlias = str               # Social media platform
    identifier
LagDays:           TypeAlias = int               # Days between shock and
    favorability poll
BlocWeight:        TypeAlias = float             # Inferred probability
    author belongs to bloc
BlocWeights:       TypeAlias = dict              # dict[BlocId, BlocWeight],
     sums to 1.0
Party:             TypeAlias = Literal["democrat", "republican"]
# Party controls which direction the pipeline optimizes.
# Democrat V_eq ~0.52-0.53; Republican V_eq ~0.49-0.51 (voter panel,
    build_constraint_spec).
# Every component that reads vote share or sets a win condition must
# receive party as an explicit argument -- never hardcode "democrat".
```

Conventions:

- Election cycles are stored as `int` in `YYYY` format. No `datetime` or `pd.Timestamp` in payload classes.
- Bloc identifiers are lowercase snake-case strings (e.g. `"african_american"`, `"evangelical"`, `"catholic"`).
- All share/weight values are floats in $[0, 1]$.

Part 4: Round-Trip Tests (`tests/test_artifact_roundtrip.py`)

Each test must: (i) instantiate a minimal valid payload, (ii) call `validate()`, (iii) serialize via `to_dict()`, (iv) confirm JSON-serializability, (v) reconstruct via `from_dict()`, and (vi) assert `obj2.to_dict() == d`.

Negative tests must confirm that duplicate cycles in `VoterPanelData.cycles` and weights not summing to 1 in `BaselinePortfolioData.weights` raise informative `ValueError`s.

Files to Create or Modify

- `electoral/artifacts.py` (expand with payload classes)
- `electoral/core/schema.py` (create)
- `electoral/core/types.py` (create; includes `Party` type alias)
- `electoral/core/io.py` (Parquet + JSON artifact I/O)
- `electoral/config.py` (`PipelineConfig` must include `party: Party` field and `target: float`; never hardcode either. Also include `pipeline_mode: str` with two valid values: `"historical"` (full rebuild, used during SRP development) and `"continuous"` (nightly incremental update, collects new shocks only, skips full voter panel rebuild). The mode controls which Prefect tasks are triggered in `pipeline.py`.)
- `electoral/stages.py` (create with one stub function per stage)

- `electoral/pipeline.py` (Prefect flow replacing hand-written DAG)
- `Justfile` (targets: `just smoke`, `just test`, `just score`, `just train`, `just deploy`, `just sample`, `just clean`, `just continuous`)
- `DOMAIN.md` (documents how to instantiate the pipeline for a new domain: what config fields to set, what data to provide, what fine-tuning schema to use. The electoral vertical is the first implementation; this file is the contract for future verticals)
- `tests/test_artifact_roundtrip.py` (create)

Pi connectivity — use Tailscale, not mDNS.

mDNS/Bonjour will not work on CMC's enterprise WiFi. University enterprise networks aggressively block multicast DNS (mDNS) at the wireless controller level to prevent broadcast storms and enforce client isolation. Devices on the same campus WiFi cannot resolve `pi.local` — the mDNS queries are silently dropped at the switch. Static IP assignments on DHCP networks are equally unreliable: DHCP leases expire, and IT security policies may quarantine unknown MAC addresses. Do not rely on either approach.

Install Tailscale on the Pi and all local machines (Week 0). Tailscale is a free zero-config VPN that assigns each device a stable `100.x.x.x` IP regardless of the underlying network. It works through NAT and enterprise firewalls via relay nodes, requires no IT coordination, and survives DHCP lease changes. After installing and authenticating on all machines, the Pi's Tailscale IP is stable and accessible from any network. Update `configs/base.json` to use the Tailscale IP: `"pi_bio_server": "http://100.x.x.x:9000"`.

Canonical Bloc Identifiers (Authoritative — Do Not Deviate)

The following table is the single source of truth for all `BlocId` strings used throughout the pipeline. Confirmed with Prof. Espinosa in the Week 1 check-in and recorded in `DECISIONS.md`.

Parallel three-stratum demographic architecture. Each stratum is independently mutually exclusive and exhaustive over ∼100% of the electorate. Every voter belongs to exactly one cell in each stratum. Strata are *parallel* — not nested — so there are no cross-tabulations required and no sparse intersection cells.

Stratum 1 — Race/Ethnicity (optimizer decision variables w_i):

Display Name	Canonical `RaceId`	Approx. share
African American	`african_american`	∼12%
Latino / Hispanic	`latino`	∼11%
Asian American	`asian`	∼5%
White	`white`	∼62%
Other	`other_race`	∼10%

Sums to 100%. White includes non-Hispanic White only.

Stratum 2 — Religion (independent stratum, weights v_R):

Display Name	**Canonical `ReligionId`**	**Approx. share**
Evangelical Protestant	`evangelical`	∼24%
Catholic	`catholic`	∼21%
Mainline Protestant	`protestant`	∼13%
Secular / None	`secular`	∼26%
Jewish	`jewish`	∼2%
Muslim	`muslim`	∼1%
Other religion	`other_rel`	∼13%

Sums to 100%.

Stratum 3 — Gender (independent stratum, weights g_G):

Display Name	**Canonical `GenderId`**	**Approx. share**
Women	`women`	∼52%
Men	`men`	∼47%
Other	`other_gender`	∼1%

Sums to 100%.

How the three strata combine. Each stratum independently estimates loyalty. Effective loyalty is a **weighted average** across all three strata:

$$\mu^{\text{eff}} = \lambda_1 \sum_i w_i \mu_i^{\text{race}} + \lambda_2 \sum_R v_R \mu_R^{\text{rel}} + \lambda_3 \sum_G g_G \mu_G^{\text{gen}}$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$ are **layer weights** calibrated from the voter panel by regressing each stratum's marginal loyalty against actual electoral outcomes across historical cycles. Typically $\lambda_1 > \lambda_2 > \lambda_3$, but the values are data-driven, not hardcoded.

The optimizer's decision variables remain w_i (the five race weights) because turnout allocation operates at the racial-bloc level. Religion and gender weights v_R and g_G are **fixed** from the voter panel — the optimizer does not rebalance religion or gender, only race.

Win condition:

$$\mu^{\text{eff}}(\mathbf{w}) \;\geq\; V_{\text{eq}}^{(P)}$$

where $\mu^{\text{eff}}(\mathbf{w})$ is computed from the three-stratum formula above. The covariance matrix $\mathbf{\Sigma}$ in the optimizer is 5×5 (one entry per racial bloc), derived from historical cycle-to-cycle variance in μ_i^{race}.

Why this resolves the intersection problem. The previous architecture required race × religion × gender cross-tabulations, producing sparse cells (Asian Muslim, Latino Jewish) that needed imputation. The parallel stratum architecture uses only marginal tables — each stratum is estimated from its own data independently. A Latino Catholic woman contributes to the Latino estimate, the Catholic estimate, and the Women estimate separately. No intersection cell is ever needed, so no imputation is required.

The additive linearity assumption is an approximation — not a solution. The paper must say so. The formula $\mu^{\text{eff}} = \lambda_1 \sum_i w_i \mu_i^{\text{race}} + \lambda_2 \sum_R v_R \mu_R^{\text{rel}} + \lambda_3 \sum_G g_G \mu_G^{\text{gen}}$ assumes demographic identities contribute independently and additively. This is a tractable approximation that will be wrong in important cases. A Latino Evangelical voter's political

behavior is not accurately reconstructed by independently summing the Latino marginal, the Evangelical marginal, and the male marginal — intersection-specific dynamics are smeared. This is a form of the **ecological fallacy** (Robinson, 1950): aggregate marginal correlations do not guarantee individual-level accuracy.

The theoretically correct alternative is Multilevel Regression with Poststratification (MRP), which models joint probability distributions across intersectional cells using hierarchical Bayesian shrinkage. MRP is not implemented here due to time and data constraints.

Required in the paper: (1) State the additive independence assumption explicitly. (2) Report the in-sample fit of the additive model on held-out cycles — if the model fits poorly, MRP is the honest alternative. (3) Cite Prof. Espinosa's own research on Latino Evangelical political distinctiveness as evidence that the marginal approach may underfit for that specific intersection.

Confirmed with Prof. Espinosa in Week 1 check-in and recorded in `DECISIONS.md`. The voter panel (ARDA, GSS, NEP) provides all required cross-tabulations: race × religion composition ($\alpha_{i,R}$), religion × race vote shares ($\mu_{i,R}$), and race × religion × gender vote shares ($\mu_{i,R}^{(F,M)}$).

Optional Overtime (Day 6) — Week 0

Use if infrastructure issues consumed more time than expected, or to get ahead.

- Begin downloading the Kavanaugh archive and 2020 election datasets to Intel Mac
- Run a test Prefect flow end-to-end with a stub artifact
- Pre-label 20 user bios per bloc for the SetFit training set
- Confirm ARDA cross-tabulation data exports at race × religion × gender level

4 Week 1: Historical Data Pipeline and Voter Panel

Objective of Week 1

Implement the data stage kernel that constructs a validated, longitudinal voter panel. By the end of Week 2:

> *Given raw survey exports from ARDA, GSS, Gallup, NEP, and Pew, the system produces a `VoterPanelData` artifact whose schema is stable, whose contents satisfy declared invariants, and whose output is deterministic under a fixed seed.*

CONCEPT: Voter panel

The voter panel is a longitudinal table of within-cell vote shares across election cycles. For each combination of race × religion × gender in each election year, it stores what fraction of that specific intersection voted Democratic. This is completely different from national vote share, which is a weighted aggregate. A White Evangelical woman might have $\mu_{i,R}^{(F)} = 0.26$ — the panel stores that cell-level figure. The panel also provides the cross-tabulations ($\alpha_{i,R}$, $\beta_{i,R}$)

needed to compute effective loyalty μ_i^{eff}. ARDA, GSS, and NEP all provide these cross-tabs from their survey methodology.

WHY THIS DESIGN CHOICE

Why store data at the race × religion × gender level rather than a simpler structure? Because the three-layer hierarchy requires the joint distribution to compute cross-tabulation weights. You cannot derive $\alpha_{i,R}$ (religion share within race i) from marginal tables alone — you need the joint. ARDA and GSS both provide multi-way cross-tabulations at this level.

Data Sources

Source	Abbreviation	Coverage
American Religion Data Archive	ARDA	2000–2024
General Social Survey	GSS	1972–2024
Gallup Poll	Gallup	2000–2024
National Election Pool	NEP	2000–2024
Pew Research Center	Pew	2004–2024

Required Panel Columns (Week 2 Minimal Set)

The panel is stored as **three independent marginal tables** — one per stratum. No cross-tabulations are needed. Each table covers ∼100% of the electorate.

Race/Ethnicity table (`panel_race.parquet`):

- `cycle` (`int`, YYYY): election year
- `race` (`str`): one of `african_american`, `latino`, `asian`, `white`, `other_race`
- `vote_share` (`float`): Democratic within-bloc vote share
- `stratum_share` (`float`): fraction of the electorate in this racial group (used to compute v_R^{race} for Stratum 1)
- `turnout` (`float`): estimated turnout rate for this cell
- `source` (`str`): e.g. `"ARDA"`, `"NEP"`

Religion table (`panel_religion.parquet`):

- `cycle`, `source`
- `religion` (`str`): one of `evangelical`, `catholic`, `protestant`, `secular`, `jewish`, `muslim`, `other_rel`
- `vote_share`, `stratum_share`, `turnout`

Gender table (`panel_gender.parquet`):

- `cycle`, `source`

- `gender` (`str`): `women`, `men`, `other_gender`
- `vote_share`, `stratum_share`, `turnout`

No cross-tabulations needed. Each table is sourced from its own marginal survey data. ARDA and GSS provide robust race, religion, and gender marginals separately; no joint distribution is required. This eliminates all sparse cell imputation problems.

Layer weight calibration. `build_constraint_spec` also estimates $\lambda_1, \lambda_2, \lambda_3$ by regressing each stratum's weighted loyalty against actual electoral outcomes across historical cycles. Store in `configs/layer_weights.json`.

Optional: marginal calibration (raking) as a consistency check. Raking (iterative proportional fitting) is a lightweight middle ground between the additive parallel stratum model and full MRP. It cycles through the three stratum marginal tables one at a time, rescaling each to match known population totals, repeating until all three margins converge simultaneously. Unlike MRP, it requires only the marginal tables you already have — no joint cross-tabulations needed.

> **CONCEPT: Raking** Given marginal totals $\{w_i^{\text{race}}\}$, $\{v_R^{\text{rel}}\}$, $\{g_G^{\text{gen}}\}$ from the voter panel, raking iteratively: (1) rescale race weights so $\sum_i \hat{w}_i = 1$ matching race marginals; (2) rescale religion weights so $\sum_R \hat{v}_R = 1$ matching religion marginals; (3) rescale gender weights so $\sum_G \hat{g}_G = 1$ matching gender marginals; repeat until convergence (typically <10 iterations). The result is a set of weights that are jointly consistent with all three marginals simultaneously, without assuming the additive independence of the parallel stratum model.

Implement in `electoral/kernels/raking.py` as an optional post-processing step on the λ-weighted outputs of `build_constraint_spec`. If raking produces loyalty estimates that differ substantially from the additive model, report both in the paper and discuss the discrepancy. A large divergence indicates the additive independence assumption is empirically poor for the given cycle — which is itself an important finding. Store the raking output alongside the additive output in `configs/layer_weights.json` under the key `"raked"`.

Required Invariants

- **Uniqueness:** each (`cycle`, `strat_id`, `source`) tuple appears at most once within each table.
- **Sorting:** rows sorted by (`cycle`, stratum column).
- **Type discipline:** `cycle` is `int` YYYY; all string columns are lowercase snake-case.
- **Finite values:** `vote_share`, `stratum_share`, and `turnout` are finite and lie in $[0, 1]$.
- **Completeness:** for each `cycle`, the sum of `stratum_share` within each table equals 1.0 within `tol=1e-6`.

Methodology

Multi-source conflict resolution. When the same (`cycle`, `bloc`) pair appears in multiple sources with different `vote_share` values, compute a weighted average using inverse standard error as weights where available, equal weights otherwise. The resolution rule and any per-source discrep-

ancies exceeding 3 percentage points are recorded in `DECISIONS.md` and flagged for the supervision check-in. Do not silently aggregate — raise on exact duplicates within the same source.

Imputation strategy. For bloc-cycle cells missing from all sources: linear interpolation between the nearest non-missing cycles for runs of 1–2 consecutive gaps; carry-forward from the previous cycle for isolated single gaps at the boundary. Imputed cells are tagged `"imputed": true` in the artifact and shown in gray on the analyst dashboard coverage matrix. No imputation is applied if more than 2 consecutive cycles are missing — those cells remain `"missing"` and are excluded from moment estimation.

Parquet output (Option A). The cleaned panel is written as a Parquet file to `data/panel/voter_panel.parquet` via `pandas.DataFrame.to_parquet(engine="pyarrow", compression="snappy")`. The `VoterPanelData` artifact `data` payload points to this path rather than embedding the table inline. DuckDB can query the Parquet directly: `duckdb.read_parquet("data/panel/voter_panel.parquet")` without loading into memory. This is 5–10× faster than JSON for downstream stages and reduces the Syncthing sync payload.

Validation gates. Before writing the artifact, run `validate_panel(df)` which enforces all four invariants. Additionally: (i) assert all 10 canonical `bloc_id` strings are present for at least one cycle; (ii) assert no `vote_share` value exceeds 0.97 or falls below 0.02 (flag as likely data error, not error out); (iii) assert national Democratic two-party vote share implied by $\sum_i w_i^{\text{turnout}} \cdot \mu_i$ is within 5 pp of the actual national result for each cycle (sanity cross-check).

Files to Create

- `electoral/kernels/data.py`
- `electoral/kernels/raking.py` (optional marginal calibration via iterative proportional fitting; post-processes stratum weights from `build_constraint_spec`; outputs `"raked"` key in `configs/layer_weights.json`)
- `electoral/data/loaders.py`
- `electoral/data/cleaning.py`
- `electoral/data/panel.py`
- `data/panel/voter_panel.parquet` (output artifact)
- `tests/fixtures/toy_panel.csv`
- `tests/test_data_cleaning.py`

Juhi does not modify `stages.py` in Week 2; kernel integration happens in Week 2 Day 4 after the cleaning logic is validated.

Optional Overtime (Day 6) — Week 1

Use to get ahead on data-intensive steps that benefit from overnight processing.

- Begin Gemini/local-LLM cleaning of the first archive batch (start overnight run)
- Pre-download additional ARDA/GSS cycles for edge-case cross-tab coverage

- Write and run the schema validation test suite end-to-end on stub data
- Document any discrepancies in voter panel source coverage in `DECISIONS.md`

5 Week 2: Baseline Voter Portfolio

Objective of Week 2

Implement the machine learning kernel that identifies the optimal steady-state demographic distribution associated with historical winning coalitions for the configured party. By the end of Week 3:

> *Given a validated `VoterPanelData` artifact and a `PipelineConfig` with `party` and `target` set, the system produces a `BaselinePortfolioData` artifact containing the demographic vote-share distribution that minimizes coalition variance while reaching* $V_{\text{eq}}^{(P)}$.

Mathematical Framework

CONCEPT: Coalition portfolio analogy

The mathematical structure here is identical to mean-variance portfolio optimization (Markowitz, 1952). In portfolio theory: assets have expected returns and a covariance matrix; you find the minimum-variance portfolio that meets a return target. In the electoral model: racial blocs have effective loyalty scores μ_i^{eff} and a covariance matrix Σ of loyalty fluctuations; you find the minimum-variance coalition (most stable) that meets the win threshold. The math does not care whether the underlying objects are stocks or voter blocs — it only sees the structure.

This project uses a **parallel three-stratum architecture**: Race/Ethnicity, Religion, and Gender each independently cover ∼100% of the electorate. No cross-tabulations are required; no intersection cells are imputed.

- **Stratum 1 — Race/Ethnicity**: coalition weights w_i over five mutually exclusive racial blocs ($i \in \{\text{aa, lat, as, wh, other}\}$). Optimizer decision variables. Sum to 1. Loyalty: $\mu^{\text{race}} = \sum_i w_i \mu_i^{\text{race}}$.
- **Stratum 2 — Religion**: fixed weights v_R over seven religious groups, from the voter panel. Optimizer does not rebalance religion. Loyalty: $\mu^{\text{rel}} = \sum_R v_R \mu_R^{\text{rel}}$.
- **Stratum 3 — Gender**: fixed weights g_G over three gender groups (women, men, other), from the voter panel. Loyalty: $\mu^{\text{gen}} = \sum_G g_G \mu_G^{\text{gen}}$.

Effective loyalty (scalar):

$$\mu^{\text{eff}}(\mathbf{w}) = \lambda_1 \mu^{\text{race}} + \lambda_2 \mu^{\text{rel}} + \lambda_3 \mu^{\text{gen}}$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$ calibrated from historical data.

The win condition:

$$\mu^{\text{eff}}(\mathbf{w}) \;\geq\; V_{\text{eq}}^{(P)}$$

How $V_{\text{eq}}^{(P)}$ is derived. $V_{\text{eq}}^{(P)}$ is the empirical winning threshold — the minimum weighted coalition share historically sufficient to win the presidency. It is computed in `build_constraint_spec` from the voter panel as follows:

1. For each election cycle c in the panel, compute the winning candidate's weighted coalition sum $S_c = \sum_i w_i^{(c)} \mu_i^{\text{eff},(c)}$ using the historical vote shares and turnout weights from that cycle.
2. $V_{\text{eq}}^{(D)} = \text{mean}(S_c \mid \text{Democrat won cycle } c)$, $V_{\text{eq}}^{(R)} = \text{mean}(S_c \mid \text{Republican won cycle } c)$.
3. These are stored in `configs/party_config.json` and loaded at runtime. Empirically, they will land in the range 0.52–0.53 for Democrats and 0.49–0.51 for Republicans, reflecting the Electoral College geography disadvantage Democrats face and the fact that Republicans won the presidency twice (2000, 2016) while losing the national popular vote.

The expected empirical range is 49–53%. Democrats typically need 52–53% of the weighted coalition share because EC geography penalises their population concentration in large blue states. Republicans can win with 49–51% because EC geography advantages them — Bush (2000) and Trump (2016) won the presidency while losing the national popular vote. If the panel produces a value outside 49–53%, check the turnout weights and bloc definitions before accepting it.

The covariance matrix $\mathbf{\Sigma}$ in the optimizer is 5×5, derived from cycle-to-cycle variance in μ_i^{race} across historical election data.

When a shock occurs, the LLM outputs delta bins per stratum. Post-shock effective loyalty: $\tilde{\mu}^{\text{eff}} = \mu^{\text{eff}} + \Delta\mu^{\text{eff}}$ where $\Delta\mu^{\text{eff}} = \lambda_1 \sum_i w_i \Delta\mu_i^{\text{race}} + \lambda_2 \sum_R v_R \Delta\mu_R^{\text{rel}} + \lambda_3 \sum_G g_G \Delta\mu_G^{\text{gen}}$. The optimizer finds $\tilde{w}_i$ maximizing $P(\tilde{\mu}^{\text{eff}} \geq V_{\text{eq}})$.

Party field. When `party = "republican"`, use $1 - \mu$ for all stratum vote shares.

The baseline portfolio solves:

$$\max_{\mathbf{w}} \; P\Big(\mu^{\text{eff}}(\mathbf{w}) \geq V_{\text{eq}}\Big) \quad \text{subject to} \quad \sum_i w_i = 1, \quad w_i \geq 0$$

A Gaussian Process classifier provides calibrated win probability estimates as a second validation signal (see Methodology below).

Methodology

The baseline pipeline has two parallel components that cross-validate each other.

CONCEPT: CVXPY

CVXPY is a Python library for convex optimization. You write the objective and constraints in near-mathematical notation — `cp.minimize(cp.quad_form(w, Sigma))` — and CVXPY automatically selects and calls an appropriate solver (SCS, ECOS, or OSQP depending on problem type). The key property of convex programs: any local optimum is the global optimum, so the solution CVXPY finds is guaranteed to be the best possible. Our minimum-variance coalition problem is convex by construction.

CONCEPT: LedoitWolf covariance

The covariance matrix Σ measures how racial-bloc loyalty returns move together historically. A standard sample covariance matrix is unreliable when you have fewer historical election cycles than blocs — it can become singular (non-invertible), causing CVXPY to crash. LedoitWolf is a regularized estimator that shrinks the sample covariance toward a structured target (typically the identity matrix scaled by the average variance), guaranteeing a positive-definite result regardless of sample size. With only 6–8 election cycles in the panel, this regularization is not optional — it is required for correctness.

Component 1: CVXPY Minimum-Variance Optimizer. After estimating $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ from the panel, CVXPY solves the QP above. The output is the demographic weight vector $\mathbf{w}^*$ that minimizes coalition volatility while guaranteeing the equilibrium threshold is met on historical data. This is the "baseline portfolio" — the starting point for all post-shock rebalancing in Week 6.

Component 2: Gaussian Process Win-Probability Classifier (Option E). A Gaussian Process classifier with a composite RBF + Matern kernel replaces XGBoost. GPs explicitly model temporal correlations — the 2020 cycle is weighted more heavily than the 2000 cycle when predicting 2024 — and provide calibrated uncertainty estimates out of the box via posterior predictive distributions. With only 6–8 training cycles, the cubic scaling of GPs is irrelevant and their uncertainty quantification is more honest for the paper's epistemic humility framing than XGBoost's confidence scores. The GP is implemented via `sklearn.gaussian_process.GaussianProcessClassifier` with a `RBF() + Matern(nu=1.5)` kernel.

Leave-One-Cycle-Out (LOCO) Validation. For each of the 6–7 election cycles in the panel, the model is trained on all other cycles and evaluated on the held-out one. Accuracy, MAE of predicted vote shares, and whether the QP remains feasible on the held-out data are all recorded in `artifacts/baseline_loco.json`. The model must perform above chance before proceeding to Week 4.

Cross-validation against prediction markets. For cycles where prediction market data is available (2004 onward via IEM, 2014 onward via PredictIt), compare the GP's predicted win probability on the held-out cycle against the IEM/PredictIt price at the time of the election. This is the strongest possible validation benchmark — if the model's baseline probability is systematically far from the market-implied probability on held-out cycles, the moment estimates or constraint bounds need re-examination before proceeding.

Constraints Module. Domain knowledge from Prof. Espinosa's published research constrains the optimization. Constraints are *cycle-parameterized*, not hardcoded — Evangelical share drifted from ∼26% of the electorate in 2000 to ∼22% in 2024, so a static minimum bound of 0.18 would force demographically impossible coalitions on earlier cycles.

The `ConstraintSpec` dataclass holds bounds as a function of cycle:

```
@dataclass(frozen=True)
class ConstraintSpec:
    # w_min[bloc_id][cycle] = minimum coalition weight for that bloc in
       that cycle
    # Derived from actual eligible voter pool fractions in the panel.
    w_min: dict[str, dict[int, float]]
    w_max: dict[str, dict[int, float]]
```

```python
def build_constraint_spec(panel_df: pd.DataFrame) -> ConstraintSpec:
    """Derive per-cycle bounds from the actual voter panel turnout
       fractions."""
    ...
```

For the post-shock (Week 6) optimizer, bounds are evaluated at the most recent cycle. For LOCO validation, bounds are evaluated at the held-out cycle. Confirmed with Prof. Espinosa during the Week 3 supervision check-in.

Files to Create

- `electoral/kernels/baseline.py`
- `electoral/models/ml_baseline.py` (moment estimation, GP classifier, LOCO validation)
- `electoral/portfolios/weights.py` (equal-weight and value-weight baselines)
- `electoral/portfolios/constraints.py` (`ConstraintSpec` dataclass)
- `electoral/portfolios/cvx.py` (baseline CVXPY solver)
- `artifacts/baseline_loco.json` (LOCO validation results)
- `docs/baseline_interpretation.md` (domain interpretation of weights)
- `tests/test_baseline.py`

Optional Overtime (Day 6) — Week 2

Use if the GP baseline consumed more time than expected, or to pre-stage Week 3 data.

- Submit the first SLURM scoring array job for any archives already cleaned
- Begin Hailo HEF compilation if not completed in Week 0
- Label an additional 50 bios per bloc for the SetFit training set
- Spot-check the constraint spec output against hand-calculated values for 2020

6 Week 3: RoBERTa Pipeline, Social Media Sentiment, and Fine-Tuning Dataset

---

Objective of Week 3

Build three parallel data collection paths — a news corpus scraper, a social media sentiment collector, and a prediction market aggregator — score news and social media with RoBERTa, correlate all signals against 14-day lagged favorability polls, and assemble a unified fine-tuning dataset. By the end of Week 4:

> *Given a shock event identifier, the system collects news articles, social media posts across four platforms, and prediction market price movements from Polymarket, PredictIt,*

Metaculus, and Manifold; scores text with RoBERTa; and produces a `SentimentData` *artifact, a* `SocialMediaSentimentData` *artifact, a* `PredictionMarketData` *artifact, and a unified fine-tuning dataset with at least 400 training examples.*

CONCEPT: RoBERTa

RoBERTa (Robustly Optimized BERT Pretraining Approach) is a transformer model pretrained on 160GB of English text. Like all transformer models, it converts input text into dense vector representations that encode semantic meaning. We fine-tune a classification head on top of RoBERTa to predict political sentiment per demographic cell: how much does this post indicate support or opposition for Party P within a given race $\times$ religion group?

The key advantage over simpler sentiment models (VADER, TextBlob): RoBERTa understands context. "This is not good for the country" reads as negative even without the word "bad." It handles sarcasm, political code-switching, and domain-specific vocabulary better than keyword-based approaches.

CONCEPT: SetFit bio classifier

SetFit is a few-shot text classification framework. "Few-shot" means it achieves high accuracy with only 100–200 labelled examples per class — important because manually labelling bios is expensive. SetFit works by: (1) converting bios to dense vectors using a sentence transformer (`all-MiniLM-L6-v2`), (2) training a lightweight logistic regression head using contrastive learning (pulling same-class bios together in embedding space, pushing different-class bios apart). The result: a classifier that maps any bio text to probability distributions over race and religion simultaneously, handling overlap correctly.

Prediction Markets as an Incentive-Compatible Belief Signal

Why prediction markets. Every other signal in this pipeline suffers from cheap talk: polls are self-reported, social media has no cost to posting, news articles are written by observers. Prediction market prices are the only signal that is *incentive-compatible* — participants lose real money when wrong, so prices represent the best-available aggregate estimate of the probability of the event, incorporating private information no survey can elicit. This is the canonical result from stochastic operations research and mechanism design (Wolfers & Zitzewitz 2004; Arrow et al. 2008).

Formally, if the market price for a contract paying \$1 iff candidate P wins is $\pi_s^P(t)$ at time t, then the belief update around shock s is:

$$\Delta\pi_s^P = \pi_s^P(t + 24\text{h}) - \pi_s^P(t - 24\text{h})$$

Critical distinction: probability vs. margin — but demographic skew is useful. Prediction markets trade win *probability* (π), not vote-share *margin* (μ). A 5-point swing in win probability cannot be directly interpreted as a 5-point swing in vote share — the mathematical category error remains regardless of who is trading. For this reason, $\Delta\pi$ is **not** used as a training feature for the vote-share estimator.

Furthermore, a prediction market price is not a measure of what traders *believe politically*

— it is their estimate of what *the national electorate will do*. When Polymarket drops after a macroeconomic shock, traders are updating their model of how the median national voter will react, not expressing their own political allegiance. A Gen Z male buying a Trump contract is forecasting national outcomes, not registering his vote intention. Treating $\Delta\pi$ as a sentiment proxy for Gen Z male voters conflates first-order personal sentiment with second-order speculative forecasting — two entirely different signals.

Two uses in the pipeline (prediction markets as calibration only):

1. **As a post-hoc calibration benchmark.** After each historical shock event, the model's Monte Carlo win probability $P(\text{win})$ is compared against $\Delta\pi_s^P$ in the audit log. Systematic divergence informs recalibration. This is calibration of model-output probability against market probability — not using market data as a training feature.

2. **As a real-time display benchmark.** The live Polymarket/Kalshi price is displayed alongside the model output as "Market consensus: 61%" vs. "Coalition model: 39%." The two numbers answer different questions: markets aggregate all public information into a win probability; the model shows how coalition *structure* must shift.

Prediction market data does not enter the training dataset. The `prediction_market` block is retained in the schema for audit log comparison only. $\Delta\pi$ is **not** stored as a demographic signal, **not** used as a training feature, and **not** used as a supervisory target for $\Delta\mu$. It is a benchmark, not a signal.

Data sources and access (post-2012 only to avoid microstructure incompatibilities between early academic markets and modern platforms):

Market	Coverage	Access	Notes
Polymarket	2020–present	Free REST API	Highest volume; CFTC-regulated
PredictIt	2014–present	Free data API	Strong 2014–2020 coverage
Metaculus	2015–present	Free REST API	Community forecasting; lower volume

IEM (1988–present) excluded: position limits incompatible with modern platform liquidity.

Coverage strategy. Use **post-2012 data only.** Pre-2012 academic markets (IEM, early PredictIt) had position limits and liquidity profiles incompatible with modern platforms; naive aggregation across eras introduces microstructure noise. For shocks 2020–2024: Polymarket + PredictIt. For shocks 2013–2020: PredictIt + Metaculus. For shocks before 2013: no market data; the `prediction_market` block is `null` in the training record. This restriction means fewer training examples have market calibration data, but those that do are internally consistent.

Multi-market aggregation. When multiple markets cover the same event, aggregate prices weighted by trading volume:

$$\bar{\pi}_s^P(t) = \frac{\sum_m V_{m,s}(t) \cdot \pi_{m,s}^P(t)}{\sum_m V_{m,s}(t)}$$

where $V_{m,s}(t)$ is the dollar volume traded on market m in the 24h window around time t. Higher-volume markets are more liquid and therefore more informative; volume-weighting is more principled than simple averaging. If volume data is unavailable (Metaculus, Manifold), assign equal weight to that market's contribution.

The core undercoverage problem is that only 6–8 historical election cycles have clean polling responses to shock events, yielding a very small fine-tuning dataset. Social media provides a high-frequency weak supervisory signal that significantly expands the effective dataset size: for each shock event, millions of posts across four platforms are available within 72 hours, stratified by platform demographic profile.

Platform-to-bloc demographic proxies:

Platform	**Default Demographic Proxy**	**Access Method**
Bluesky	Chronically online, strongly liberal (not broadly secular/-diverse — skews far-left, post-Twitter migration)	AT Protocol firehose (free, no cap)
Apify (X scraper)	High-edu urban, Evangelical, Catholic	Free tier scraper; 500 results/run
Telegram	Far-right, MAGA-aligned, Evangelical	Historical archive only (USC dataset)
Facebook	Older, Catholic, mainline Protestant	Meta Content Library*
Truth Social	White Evangelical, conservative	Direct scrape (Mastodon fork)
Reddit	Broad ideological range; subreddit-level proxies (r/Christianity, r/Catholicism, r/LatinoPeopleTwitter, r/BlackPeopleTwitter, r/-Conservative, r/progressive, r/exchristian)	Pushshift archive (pre-2023) + Reddit API free tier
Discord	Young male, politically organized communities; server-level proxies	Discord-Unveiled-Compressed (HuggingFace; 2.05B messages, 3,167 servers, 2015–2024)

*Requires academic approval — apply in Week 1.

Bluesky and Apify run simultaneously; archives supplement both.

Prediction markets (Polymarket, Kalshi) are calibration benchmarks only, not platform proxies.

These platform-level proxies are the **fallback** when a post author cannot be classified by bio analysis. The primary demographic assignment method is probabilistic bio-based inference, described below.

CONCEPT: Prediction markets: calibration benchmarks, not demographic proxies

Polymarket and Kalshi prices aggregate what traders estimate *the national electorate will do* — not what those traders themselves believe politically. When a market drops after a macroeconomic shock, traders are updating their model of how the median national voter will react, not expressing their own allegiance. A prediction market price is second-order forecasting of national sentiment, not first-order expression of the trader's own ideology. Treating $\Delta\pi$ as Gen Z male sentiment conflates these two distinct phenomena.

$\Delta\pi$ is used only as a post-hoc calibration benchmark: after each shock resolves, the model's Monte Carlo win probability is compared against market prices in the audit log. Systematic divergence informs recalibration. The live market price is also displayed alongside the model output in the app so users can compare the model's coalition-structure prediction against market consensus on win probability. Prediction market data does not enter the training dataset in any form.

Probabilistic Bio-Based Demographic Inference

Social media platforms that provide post author profile objects enable a probabilistic demographic inference layer. The bio classifier now outputs a **two-layer** probability vector per post: Layer 1 (race) and Layer 2 (religion), plus a Layer 3 gender signal. This replaces the previous flat `bloc_weights` dict.

Output per post: a structured object:

- `race_weights`: probability distribution over {`african_american, latino, asian, white`}
- `religion_weights`: probability distribution over {`evangelical, catholic, protestant, secular, jewish, muslim`}
- `gender_signal`: `"F"`, `"M"`, or `null`

The scorer then computes per-cell sentiment scores $s_{i,R}^{(G)}$ for each race × religion × gender combination. These collapse to per-racial-bloc sentiment scores via the $\alpha_{i,R}$ and $\beta_{i,R}$ weights from the voter panel, producing the `social_roberta_scores` field in the fine-tuning dataset.

Why this matters. A flat bloc-weight classifier cannot distinguish a Latino Catholic's response to a shock from a White Catholic's response, even though both have high `catholic` weight. The two-layer output preserves this distinction and allows the LLM to learn race-specific religious responses. It also handles overlap correctly: a user whose bio reads "Proud Latina Catholic" receives high `latino` race weight and high `catholic` religion weight simultaneously, without double-counting in the optimizer.

Three-stage inference pipeline:

1. **Keyword matching (high precision, fast).** A curated lexicon — built from Prof. Espinosa's published research on linguistic and cultural markers per bloc — maps bio keywords and phrases to bloc probabilities. Examples:
   - `"evangelical"`, `"born again"`, `"Southern Baptist"` → `evangelical`
   - `"Latino"`, `"Chicano"`, bio written in Spanish → `latino`
   - `"Jewish"`, `"#JewishTwitter"`, bio in Hebrew → `jewish`
   - `"Muslim"`, `"Alhamdulillah"`, bio in Arabic → `muslim`

   Users with clear keyword matches are assigned directly. Coverage is typically 20–40% of politically active users.

2. **SetFit classifier (Option B).** Replace k-NN with a SetFit classification head trained via contrastive learning on the labelled bios. SetFit uses `sentence-transformers/all-MiniLM-L6-v2` as the embedding backbone and trains a

logistic head with as few as 8 examples per class via in-batch negatives. Run `setfit.Trainer` on the labelled dataset; evaluate on a 20% held-out split.

Hardware boundary (important): only the embedding backbone (`all-MiniLM-L6-v2`) is compiled to HEF and runs on the Pi NPU. The SetFit classification head is a scikit-learn logistic regression layer and *cannot* be compiled to HEF — the Hailo SDK only accepts ONNX transformer graphs. The head runs on the Pi's ARM CPU using the NPU-generated embeddings as input features. This two-stage split is the correct and only viable architecture: NPU handles the expensive embedding pass (sub-ms per bio); CPU handles the cheap logistic inference on top.

3. **Language signal (fallback only, not multiplicative).** The post `lang` field (present in all platform schemas) is used *only when both the keyword lexicon and SetFit classifier return no signal* (i.e., the bio is empty, missing, or below the SetFit confidence threshold). It is *not* multiplied by the bio-derived weights — doing so would assume statistical independence between bio language markers and tweet language, which is false (a Spanish-language bio already triggers the `latino` keyword match).

   Do not use hard 1.0 language assignments. Mapping `"es"` $\rightarrow$ `latino` $= 1.0$ flattens enormous diversity into a monolithic proxy: a Puerto Rican Catholic conservative, a Mexican secular progressive, and a Cuban-American Evangelical all write in Spanish but have very different political profiles. A hard assignment also breaks the three-layer race $\times$ religion $\times$ gender architecture by collapsing to a single bloc ID.

   Instead, the language fallback assigns a **probability distribution** over race $\times$ religion cells informed by census and Pew data on each linguistic community's demographic composition. Store in `configs/language_priors.json`:

   - `"es"` $\rightarrow$ {latino/catholic: 0.38, latino/protestant: 0.22, latino/secular: 0.18, latino/evangelical: 0.14, white/catholic: 0.05, ...} (from Pew Hispanic Religion Survey)
   - `"ar"` $\rightarrow$ {white/muslim: 0.45, african_american/muslim: 0.35, asian/muslim: 0.15, ...} (from Pew Muslim American Survey)
   - `"he"` $\rightarrow$ {white/jewish: 0.85, asian/jewish: 0.05, ...} (from Pew Jewish American Survey)
   - `"zh"`/`"ko"`/`"vi"` $\rightarrow$ {asian/secular: 0.35, asian/protestant: 0.28, asian/catholic: 0.22, ...}

   These priors are imprecise but dramatically more accurate than a hard 1.0. Log the fallback rate per language and report in the paper's methodology section so readers can assess the precision.

Priority order: keyword match > SetFit classification > language prior > platform proxy.

Exclude language-fallback posts from both mean and covariance estimation. When the language fallback fires at scale, all affected posts receive the same fixed Pew prior, forcing artificial lockstep correlation between religion sub-groups within a language community. The previous design excluded these posts from Σ_Δ but used them for mean μ

estimation. This creates a **bifurcated data reality**: μ is estimated from a large volume of artificially correlated posts, while Σ_Δ is estimated from a smaller subset of bio-classified posts. The mean-variance optimizer assumes μ and Σ come from the same generating process — using different corpora for each violates this assumption and produces internally inconsistent inputs to the optimizer.

Correct approach: Treat language-fallback posts as a held-out *validation* set only — not as training signal for either μ or Σ_Δ. After the main estimation is complete, compare the language-prior-based sentiment signal against the bio-classified signal as a sanity check. If they diverge substantially, investigate why before accepting the final model. Tag every language-fallback post with `inference_method: "language_prior"` and exclude these from all estimation. Report the fallback rate and validation comparison in the paper.

Output per post: a `bloc_weights` dict keyed by the 10 canonical bloc IDs, summing to 1.0, attached to each post record before RoBERTa scoring. Posts from users with unclassifiable bios fall back to the platform-level proxy (uniform weight across the platform's default demographic). The fallback rate and per-bloc coverage are reported in the paper.

Weighted elasticity aggregation: rather than computing a single platform-level elasticity score, the scorer computes a per-bloc elasticity score by weighting each post's sentiment by its author's `bloc_weights`:

$$\varepsilon^{\mathrm{X}}_{i,s} = \frac{\sum_u p_{u,i} \cdot \mathrm{sentiment}(u,s)}{\sum_u p_{u,i}}$$

where $p_{u,i}$ is the inferred probability that user u belongs to bloc i, and `sentiment`(u,s) is the RoBERTa score for user u's posts about shock s. This produces 10 distinct elasticity estimates per platform, one per bloc, rather than a single platform-level signal.

The lagged correlation architecture is the key methodological contribution. Platform sentiment at shock time t is correlated against favorability poll shifts at $t + 14$ days, producing a measured elasticity coefficient per platform per shock category. This is framed in the paper as *platform-mediated elasticity estimation*, not direct bloc measurement — an important epistemic distinction for reviewers.

$$\varepsilon^{\mathrm{platform}}_{p,s} = \mathrm{Corr}\Big(\mathrm{sentiment}_{p,s}(t),\ \Delta\mathrm{favorability}_{b(p)}(t+14)\Big)$$

where $b(p)$ maps platform p to its closest demographic bloc proxy.

CONCEPT: Hailo NPU

The Raspberry Pi 5 with the AI HAT+ contains a Hailo NPU (Neural Processing Unit) — a dedicated inference chip that runs transformer forward passes at sub-millisecond latency with very low power draw. We compile the `all-MiniLM-L6-v2` embedding backbone to the Hailo HEF format (the chip's native binary) using the Hailo SDK. The NPU runs the embedding pass; the Pi's ARM CPU runs the SetFit logistic head. The Pi is always on and classifies bios in real time as posts arrive via Syncthing.

Critical constraint: only ONNX-exportable transformer graphs can be compiled to HEF. Scikit-learn models (the SetFit logistic head) cannot. Do not attempt to compile the full Set-

Fit pipeline — the SDK will fail silently. CPU fallback is controlled by the `pi_npu_enabled` flag in `configs/base.json`.

WATCH OUT

Syncthing is installed on the four *local* machines only (M5, Intel Mac, Pi). Never install Syncthing on the HPC. HPC compute nodes are ephemeral — they are allocated for a job, run it, and are reclaimed. Any background daemon like Syncthing will be killed silently when the node is reclaimed. HPC data transfer uses explicit `rsync` commands before each SLURM job submission.

Unified Fine-Tuning Dataset Schema

Each training example fuses news and social signals per stratum. The output contains three independent delta-bin dictionaries — one per stratum — matching the parallel three-stratum architecture.

```
{
  "input": {
    "event_description": "...",
    "party":             "democrat",
    "news_roberta_scores_race": {
      "african_american": -0.12, "latino": -0.08, "asian": -0.04,
      "white": -0.31, "other_race": -0.10
    },
    "news_roberta_scores_religion": {
      "evangelical": -0.61, "catholic": -0.18, "protestant": -0.15,
      "secular": +0.04, "jewish": -0.02, "muslim": +0.01, "other_rel":
         -0.05
    },
    "news_roberta_scores_gender": {
      "women": -0.22, "men": -0.08, "other_gender": -0.05
    },
    "social_roberta_scores_race":     { ... },   # same keys, bio-weighted
    "social_roberta_scores_religion": { ... },
    "social_roberta_scores_gender":   { ... },
    "prediction_market": {         # metadata only, not a training feature
      "pre_shock_prob": 0.54, "post_shock_24h": 0.48,
      "delta_prob": -0.06, "volume_usd": 125000, "sources": ["polymarket"]
    },
    "bio_coverage":      0.34,
    "historical_analog": "2016_fbi_letter",
    "shock_category":    "Electoral Surprise",
    "synthetic":         false,
    "weight":            1.0
  },
  "output": {
    "delta_bins_race": {
      "african_american": "slight_neg", "latino": "mild_neg",
      "asian": "neutral", "white": "mod_neg", "other_race": "slight_neg"
    },
    "delta_bins_religion": {
      "evangelical": "strong_neg", "catholic": "mod_neg",
```

```
      "protestant": "mild_neg", "secular": "neutral",
      "jewish": "slight_neg", "muslim": "slight_neg", "other_rel": "
         slight_neg"
    },
    "delta_bins_gender": {
      "women": "mod_neg", "men": "slight_neg", "other_gender": "neutral"
    },
    "delta_eff": -0.031   # scalar; pipeline verifies against weighted
       stratum sum
  }
}
```

For non-gendered shocks, gender bins may be identical across women/men. For non-religious shocks, all religion bins may be identical. The pipeline computes `delta_eff = lambda_1*sum(w*delta_race) + lambda_2*sum(v*delta_rel) + lambda_3*sum(g*delta_gen)` and verifies it matches the stored scalar before accepting the training example.

Prediction market data is not a training input. The `prediction_market` block is metadata for audit log comparison only.

All training examples receive equal base weight.

Platform Access Notes

X (Twitter) — Archive-First Strategy. The X Basic paid API ($100/month) is not used. Instead, Twitter data is sourced entirely from free public archives for historical shocks and Apify for any live collection needed:

1. **Historical archives (primary volume).** Free public datasets cover the most important shock events directly — 56M Kavanaugh tweets (Pushshift), 3.5M 2020 election tweets (IEEE DataPort), full 2020 election dataset (Kaggle), MeToo tweets (Data.world), Congress tweets (GitHub). These are downloaded once and processed offline. No API needed.
2. **Apify X Scraper (live collection).** Free tier at `apify.com` gives 500 results/run for any shock event not covered by archives. The X demographic profile (high-education urban, Evangelical and Catholic representation) is captured through Apify without any subscription cost.
3. **Bluesky AT Protocol (primary live).** Free firehose covers the secular, diverse, and younger demographic slice that complements Apify.

Meta Content Library. Apply at `developers.facebook.com/docs/content-library`. **Apply in Week 1** — approval takes 2–4 weeks. Provides Facebook post access for academic research. If approval does not arrive by Week 4, fall back to public page reaction counts (likes, angry, sad) as a valence-only signal.

Truth Social. No official API. Mastodon-compatible endpoint at `truthsocial.com/api/v1/timelines/public`. Collect with rate-limited requests. Small user base means lower post volume; weight this platform's signal accordingly.

Pre-Collected Historical Archives

Several public datasets contain full tweet objects (text, author metadata, timestamps) covering shock events directly relevant to the registry. These bypass all API rate limits entirely and provide

orders-of-magnitude more volume than live collection for the specific events they cover. They are loaded by `electoral/nlp/archive.py` and normalised into the same `posts.jsonl` schema as live collectors — the scorer is blind to source.

Dataset	Volume	Shock events / use	Bio?	Source
Tier 1 — Full objects, directly usable				
Kavanaugh tweets	56M	Moral/Scandal (Sept–Oct 2018)	Check	Pushshift (#28)
2020 US Election (Kaggle)	—	2020 cycle, Oct 15–Nov 4	Likely	Kaggle (#62)
IEEE 2020 Election	3.5M	2020 cycle, July–Aug	Likely	IEEE DataPort (#54)
2018 Texas Debate	—	Electoral Surprise (Sept 2018)	Check	UNT Lib. (#44)
#MeToo tweets	—	Moral/Scandal (Oct 2017+)	Likely	Data.world (#92)
Congress tweets	Ongoing	All cycles, legislator context	N/A	GitHub alexlitel
Polarization corpus	—	All cycles; bio classifier training	**Yes**	Kiran et al.
SMAPP NYU elections	—	US elections, multi-cycle	Likely	SMAPP GitHub
UW ResearchWorks	—	Political events (verify content)	Check	UW Lib. (b88ba40a)
USC Telegram 2024	—	2024 election; Evangelical/MAGA	**No**	leonardo-blas (USC)
Truth Social 2024	—	2024 election; Evangelical/conservative	**No**	kashish-s (GitHub)
Reddit (Pushshift)	Billions	All cycles pre-2023; subreddit-level	**No**	Pushshift / AcademicTorrents
Discord-Unveiled	2.05B msg	All cycles 2015–2024; server-level; young male	**No**	SaisExperiments/ Discord-Unveiled-Compressed (HF)
Tier 2 — Useful for scorer / demographic training; bio uncertain or video-based				
Political tweets (Kaggle)	80k	General political; party labels	Check	kaushiksuresh147
Political partisanship	—	Dem/Rep stance labels	Check	lingshuhu (Kaggle)
Political tweets + gender	—	Women bloc demographic signal	Check	lingshuhu (Kaggle)
PoliticalTweets (HF)	—	Stance labels; RoBERTa training	Unknown	Jacobvs/HuggingFace
US Politicians tweets	—	Legislator accounts	N/A	mrmorj (Kaggle)
TikTok 2024 election	—	2024 election; younger/Latino/Asian	**No**	gabbypinto (GitHub)
Tier 2 — Format unconfirmed; check repo before downloading				
X 2024 election	—	2024 cycle; all blocs	Check	sinking8 (GitHub)
US Pres Elections 2020	—	2020 cycle; all blocs	Check	echen102 (GitHub)
Bot filter — Blocklist only, not training content				
Twitter bots (political)	—	Known bot accounts to exclude	N/A	khanradcoder (Kaggle)
ID-only — Require API rehydration; not usable at current budget				
VoterFraud 2020	7.6M	2020 election integrity	**IDs**	arXiv (#2)
Harvard 2016 election	280M	Entire 2016 cycle	**IDs**	Harvard Dataverse

Schema-check before use: confirm `text`, `created_at`, `lang` fields present.

Telegram datasets — new platform, high demographic value. The USC Telegram 2024 election datasets (`leonardoblas/usc-tg-24-us-election` and the HuggingFace distilled version) cover a platform currently absent from the pipeline. Telegram channels are the primary communication medium for far-right, MAGA-aligned, and Evangelical communities — the demographic slice that is hardest to capture via Bluesky (too far-left and non-representative) or Truth Social (too small). These datasets provide historical Telegram message objects for the 2024 election cycle. Because Telegram messages have no user bio field, the platform-level proxy (Evangelical/conservative Protestant) is applied to all posts from political Telegram channels. Add a `TelegramLoader` to `archive.py` that normalises Telegram message objects into the standard `posts.jsonl` schema.

TikTok dataset — use caption text, apply youth/diverse proxy. The `gabbypinto/US2024PresElectionTikToks` dataset contains TikTok video metadata including caption text and hashtags for 2024 election content. TikTok demographics skew strongly toward Gen Z, Latino, and Asian American voters — blocs currently underweighted in live collection. No user bio field is available; apply the platform-level proxy (Gen Z, Latino, Asian American) uniformly. Load caption text as the `text` field in `posts.jsonl`. Treat as supplementary signal for the secular and diverse blocs rather than a primary source.

Format-unconfirmed datasets (sinking8, echen102). Visit each GitHub repo before Week 4 Day 1 and run the standard schema check. If tweet objects are present, load as Tier 1. If ID-only, add to the rehydration backlog (not usable at current budget). Document findings in `data/archives/README.md`.

Reddit — subreddit-level demographic proxies. Reddit adds something no other platform in the stack provides: substantive long-form political discussion from self-selected communities that are considerably more demographically diverse than Twitter or Bluesky. Crucially, subreddit membership is a *much more defensible* demographic proxy than platform-level proxies, because communities like `r/Catholicism`, `r/BlackPeopleTwitter`, and `r/LatinoPeopleTwitter` are explicitly identity-defined.

Key subreddits and demographic proxies:

Subreddit	Bloc proxy	Notes
r/Christianity, r/Catholicism	White/Catholic, Protestant	Mainstream Christian discourse
r/exchristian	Secular (deconversion)	Strong signal for secular bloc shifts
r/Conservative, r/Republican	White/Evangelical, Protestant	Conservative political discussion
r/progressive, r/politics	Secular, diverse	Left-leaning; broad demographic
r/BlackPeopleTwitter	African American	Self-selected; culturally specific
r/LatinoPeopleTwitter, r/LatinoTwitter	Latino	Self-selected identity community
r/Jewish, r/Judaism	Jewish	Direct bloc signal
r/islam, r/Muslim	Muslim	Direct bloc signal
r/Christianity (evangelical flair)	Evangelical	Filter by post flair where available

Data access: The Pushshift Reddit archive covers posts and comments through early 2023 and is available via AcademicTorrents (`academictorrents.com`). For 2023–present, use the Reddit API free tier (100 requests/minute; OAuth required). Download subreddit-specific dumps rather than the full corpus — filter to the shock date window and relevant subreddits before downloading.

No bio field: Reddit posts and comments do not include user bio text. Apply the subreddit-level proxy directly (as in the table above) rather than running bio inference. The subreddit proxy is more defensible than most platform-level proxies because community membership is explicit. Tag each post with `inference_method: "subreddit_proxy"` and `subreddit: "r/Catholicism"` etc. for traceability. Exclude subreddit-proxy posts from Σ_Δ covariance estimation for the same reason language-prior posts are excluded: the proxy forces artificial within-subreddit correlation. Use for mean estimation only.

Discord — young male demographic signal, server-level proxies. Discord-Unveiled-Compressed (`SaisExperiments/Discord-Unveiled-Compressed` on HuggingFace; arXiv:2502.00627) provides 2.05 billion messages from 4.74 million users across 3,167 public Discord Discovery servers, spanning 2015–2024. Collected via Discord's public API with anonymization per ethical guidelines — publishable without ToS concerns. Structured JSON files.

Why Discord for this project. Discord Discovery servers are self-listed public communities. The dataset covers organized political, religious, and identity communities that provide direct demographic signal. Discord skews young and male — the same cohort that over-indexes on prediction markets and is currently underrepresented in the pipeline. Unlike prediction markets, Discord gives you *expressed opinion* rather than probabilistic forecasting of national outcomes.

Server-level demographic proxies. No user bio field exists on Discord. Apply server-level proxies based on server name, category, and topic tags. Filter the 3,167 servers to politically relevant communities before downloading — the full dataset is large; you need only the political/religious/identity servers. Key server categories and their bloc proxies:

Server type	Bloc proxy	Notes
Conservative/MAGA political servers	White/Evangelical, Protestant	Filter by server name/category
Christian community servers	Evangelical, Catholic, Protestant	High demographic precision
Secular/atheist servers	Secular	Direct identity signal
Latino/Hispanic culture servers	Latino	Self-selected identity
Black culture/community servers	African American	Self-selected identity
Asian culture/community servers	Asian American	Self-selected identity
Libertarian/crypto/finance servers	White/secular young male	Overlaps Polymarket demographic

Practical note. Download only the servers in your filtered list, not the full 3,167-server corpus. Save to `data/archives/discord/`. Tag each message with `inference_method: "server_proxy"` and `server_name` for traceability. Exclude from Σ_Δ covariance estimation (same reasoning as Reddit and language-prior posts — server proxy forces artificial within-server correlation). Use for mean sentiment estimation only.

Pre-Collected News Archives

The live scraper (`scraper.py`) collects full article text going forward from targeted outlets. For historical shock events that occurred before the SRP began, the scraper cannot be run retroactively. The Webhose free news dataset (`github.com/Webhose/free-news-datasets`) fills this gap: a static dump of full article text crawled from thousands of news sites, with publication date, site URL, language, and category labels. It provides historical news coverage for shock events from earlier election cycles where the scraper has no data.

Dataset	Volume	Use	Full text?	Source
Tier 1 — Local news; primary historical archive				
3DLNews2	8M+ URLs	1995–2024; all 50 states; 14k local outlets	HTML	W&M NewsLab (GitHub)
Tier 2 — General news; secondary historical archive				
Webhose free news	100k+	Multi-outlet; full text pre-extracted	**Yes**	GitHub (Webhose)

3DLNews requires HTML fetch and parse; Webhose is pre-extracted. Both write to `rawdata/articles/`.

3DLNews2 — why local news matters. Presidential elections are won and lost at the local level. Your live scraper hits national outlets (NYT, WaPo, Fox, CBN, Univision), but the communities that constitute the 10 canonical blocs consume local news. A Catholic parish in suburban Pennsylvania reads the Pittsburgh Post-Gazette; an Evangelical congregation in rural Georgia reads the Augusta Chronicle. 3DLNews2's coverage of 14,000 local outlets across all 50 states over three decades provides demographic signal no national outlet can replicate. The dataset is URLs with HTML text — download the URL metadata to the Intel Mac, filter by date window and state *before* fetching, then run the HTML fetch-and-parse as a SLURM array job on HPC scratch (see Archive Storage Strategy above). Never store raw HTML locally. Tag with `source="3dlnews"`.

Webhose vs. live scraper. The live scraper is targeted and ongoing — it runs on the Intel Mac, hits specific outlets chosen for their demographic reach (Christianity Today, CBN, Univision, NYT, WaPo, Fox), and collects full article text around any shock event going forward. Webhose is a static secondary supplement: useful for events before the SRP, broad multi-outlet coverage, pre-extracted text. Both feed into `rawdata/articles/` in the same schema; the RoBERTa scorer is blind to source. Use `source="webhose"` to tag Webhose articles separately so their contribution can be reported in the methodology section.

Missing bio field handling. Pre-collected datasets frequently omit the author `user.description` field to reduce file size. The loader checks for its presence: if found, the bio classifier runs normally; if absent, `bloc_weights` falls back to the platform-level proxy (uniform across the platform's default demographic). The fallback rate is logged per archive and reported in the paper. This is the same fallback path used for live collectors when bios are empty.

Source tagging. Every post loaded from an archive receives `"source": "archive"` and `"archive_id": "{dataset_slug}"` in its record. This allows the scorer to track archive vs. live contributions to the training dataset and report them separately in the methodology section.

Data Volume, Sampling Strategy, and LLM-Assisted Cleaning

You have more data than you need — and that is a problem. The combined archive corpus (Kavanaugh 56M, 2020 election datasets, MeToo, Telegram, TikTok, and others) runs to hundreds of millions of posts. Running a RoBERTa forward pass on all of them is not feasible: 56M Kavanaugh tweets at ∼50ms per inference = 32 days of serial GPU time. The HPC can parallelise this, but the real bottleneck is signal-to-noise ratio. Processing every post adds noise from off-topic content, spam, and bot traffic that survives the blocklist. The right strategy is **stratified sampling followed by LLM-assisted cleaning before the RoBERTa scorer sees any data**.

Step 1 — Stratified sampling (`scripts/sample_archives.py`, runs on HPC). For each archive dataset and each shock event, draw a stratified sample of up to 200,000 posts using three strata:

1. **Temporal strata.** Sample proportionally by day across the shock window (e.g. 3 days before, day-of, 3 days after, 7 days after) so the full arc of public reaction is represented rather than just the peak day.
2. **Demographic strata.** After bio classification runs, sample proportionally by inferred bloc so all 10 demographic groups are represented. Without this, high-volume blocs (secular, women) drown out low-volume blocs (Jewish, Muslim).
3. **Sentiment strata.** Oversample posts at the extremes of a fast keyword-based sentiment score (very positive and very negative). Extreme examples are more informative for fine-tuning than neutral ones.

Total target: ∼200k posts per major dataset, ∼5k per shock event per platform. A 25-node SLURM array job (one node per shock event) completes this in under 2 hours on the HPC.

Step 2 — LLM-assisted cleaning (`scripts/clean_with_llm.py`, runs on M5 using a **local open-weight model** for reproducibility).

> **Do not use Gemini Pro or any cloud API for the cleaning step.** Cloud-hosted LLMs undergo continuous silent weight updates, safety classifier tweaks, and prompt-routing changes. A prompt processed in June 2026 will not produce identical output in July 2026 regardless of temperature settings. Because cleaning output feeds directly into the RoBERTa scorer and the fine-tuning dataset, any non-determinism here propagates through the entire pipeline and renders the downstream RNG contract meaningless.
>
> **Use a locally-hosted open-weight model whose exact weights can be frozen and archived with the replication package.** Recommended: `Qwen2.5-7B-Instruct` or `Mistral-7B-Instruct-v0.3` run via `mlx_lm.generate` on the M5 (48GB unified memory handles 7B at full precision comfortably). Record the exact HuggingFace revision hash used in `DECISIONS.md` and include it in the replication package so future researchers use the identical weights.

The local model handles four cleaning tasks that rule-based filters cannot:

1. **Off-topic detection.** Flag and remove posts that mention the shock keyword but are not about the political event (e.g. tweets about Brett Kavanaugh's baseball coaching during the confirmation hearing). Prompt: *"Is this tweet about the political confirmation of Brett Kavanaugh? Reply yes or no."* Drop all "no" responses.

2. **Spam and low-information filtering.** Remove posts matching patterns that survive the bot blocklist but add no signal: pure retweet text, copy-paste chain tweets, emoji-only posts, news article URLs with no commentary.

3. **Text normalisation.** Expand political abbreviations (SCOTUS, POTUS, MAGA), normalise hashtag formatting, strip URLs and `@mentions` for cleaner RoBERTa input.

4. **Cross-platform deduplication.** The same post sometimes appears in multiple archives. Deduplicate on `(user_id, text_hash)` before scoring.

At ∼8 tokens/second on M5 MLX at 50 posts per prompt, 200k posts takes roughly 3–4 days per archive. **Batch process the most important archives during Weeks 2 and 3** while other pipeline components are being built.

Full pre-processing pipeline:

1. `scripts/sample_archives.py` on HPC — stratified sample per archive
2. `scripts/clean_with_llm.py` on M5 — LLM cleaning and dedup
3. `merge_posts(shock_id)` Prefect task — concatenate across sources
4. `scripts/score_array.sh` on HPC — RoBERTa forward pass on clean sample
5. `build_finetune_dataset()` — assemble JSONL training records

Selection Bias Mitigation

Users who post about a political shock are not representative of all platform users. Compute a **baseline sentiment score** for each platform in the 7 days *prior* to the shock using identical keywords (minus the shock-specific terms). Subtract the baseline from the shock-period score to isolate the incremental reaction:

$$\Delta\text{sentiment}_{p,s} = \text{sentiment}_{p,s}^{\text{shock}} - \text{sentiment}_{p}^{\text{baseline}}$$

This baseline-adjusted score is what enters the elasticity regression and the fine-tuning dataset. Document this adjustment explicitly in the paper's methodology section.

Social media is a weak signal, not ground truth — report it as such. Political social media is dominated by hyperpartisan, highly engaged users. The actual voting public operates on a flatter distribution. This pipeline uses social media sentiment as a *weak supervisory signal* to augment a very small historical dataset (25+ events) — not as a direct proxy for voter demographics. The claim is limited: "politically engaged members of each demographic responded in approximately this direction to this shock," not "the electorate responded."

Three specific biases must be acknowledged in the paper:

- **Endogeneity of political shocks.** Shocks do not hit an equilibrium vacuum. They are endogenous to existing media cycles, selective exposure, and asymmetric polarization. The 7-day pre-shock baseline subtraction partially isolates the incremental reaction but cannot fully separate the shock signal from the pre-existing media environment.

- **Algorithmic amplification.** Platform ranking algorithms up-weight high-animosity con-

tent. RoBERTa scores reflect this algorithmically amplified distribution, not organic ideological reaction. The baseline subtraction partially mitigates ambient amplification but does not eliminate it.

- **Bio-classifier coverage bias.** Only politically active users with explicit identity markers in their bios will be correctly classified by the SetFit classifier. Moderate or non-identifying users fall back to language priors or platform-level proxies. This systematically overrepresents activated viewpoints in the training signal.

Both limitations reduce to the same conclusion: the social media signal captures the reactions of *activated* demographic subgroups more reliably than it captures the median voter. Treat it as directional evidence, not as a calibrated empirical measurement.

Know your Polymarket/Kalshi participant base. Polymarket and Kalshi participants skew heavily toward Gen Z and younger male users — the same cohort that over-indexes on crypto, DeFi, and prediction market culture. This is a demographic *feature*: $\Delta\pi$ on these platforms gives you directional signal about how politically engaged young men are updating their beliefs, which is useful the same way that Telegram gives you signal about Evangelical/MAGA communities and Facebook gives you signal about older Catholic and mainline Protestant voters.

However, it also means the "market consensus" displayed in the app reflects one specific demographic slice, not the full electorate. Label it accordingly in the UI: "Market signal (Gen Z male skew): X%" rather than just "Market consensus: X%" so users understand whose beliefs they are seeing. PredictIt has broader domestic age distribution and is a better calibration reference for older-skewing shock events.

Files to Create

- `scripts/sample_archives.py` (HPC script: stratified sampling from raw archives by temporal strata, demographic strata, and sentiment extremity; target 200k posts per dataset; submit as SLURM array)
- `scripts/clean_with_llm.py` (M5 script: local open-weight model off-topic detection, spam filtering, text normalisation, and cross-platform deduplication on the sampled dataset before RoBERTa scoring; uses Google AI Studio free tier)
- `electoral/kernels/sentiment.py` (orchestrates all three pipelines)
- `electoral/nlp/scraper.py` (news corpus, runs on Intel Mac; targeted outlets: Christianity Today, CBN, Univision, NYT, WaPo, Fox)
- `electoral/nlp/news_loader.py` (`NewsArchiveLoader`: loads articles from pre-collected news archives and normalises them into the same `articles.jsonl` schema as the live scraper. Handles two sources: (i) **3DLNews2** (`data/archives/news/3dlnews/`) — fetches HTML from each URL, strips to plain text, filters by date window and optionally by state or outlet name; tags `source="3dlnews"`; (ii) **Webhose** (`data/archives/news/webhose/`) — pre-extracted text, filter by date and URL; tags `source="webhose"`. Both enforce 100-word minimum.)
- `electoral/nlp/social.py` (**platform-agnostic collector architecture**): a

`SocialCollector` abstract base class. Concrete subclasses: `BlueSkyCollector` (primary live, AT Protocol, free, no cap), `ApifyCollector` (secondary live, free tier, X demographic profile), `TruthSocialCollector` (Mastodon-compatible, free), `FacebookCollector` (Meta Content Library or reaction-count fallback). No `XCollector` — X Basic API is not used; X data comes from free archives (Kavanaugh, 2020 election) and Apify scraping. All subclasses write identically-formatted `posts.jsonl`; the scorer is blind to platform. **Note: Twint and Snscrape are both dead — Twitter blocked both in April 2023. Do not use them.**)

- `electoral/nlp/archive.py` (`HistoricalArchiveLoader`): loads pre-collected datasets and normalises them into the same `posts.jsonl` schema as live collectors. Handles missing `user.description` by falling back to platform proxy. Tags each post with `source="archive"` and `archive_id`. Accepts a `bot_blocklist: set[str]` of known bot user IDs; any matching post is silently dropped with a count logged per archive. Includes `TelegramLoader` (normalises Telegram message objects; applies Evangelical/conservative proxy as platform-level default) and `TikTokLoader` (normalises TikTok caption text; applies Gen Z/Latino/Asian proxy).
- `electoral/markets/collector.py` (prediction market price collector: Polymarket, PredictIt, Metaculus, Manifold, IEM; loads contract IDs from `configs/market_contracts.json`; outputs `PredictionMarketData` artifacts)
- `electoral/markets/aggregator.py` (volume-weighted multi-market price aggregation; live query for web app market prior)
- `electoral/nlp/bio_classifier.py` (SetFit + keyword lexicon + contrastive-trained head; CPU fallback for non-Pi environments)
- `electoral/nlp/scorer.py` (RoBERTa with bloc-weighted aggregation; caches raw embeddings to `data/embeddings/` Parquet)
- `electoral/nlp/elasticity.py` (regression + unified dataset assembly)
- `scripts/compile_hailo.py` (export `all-MiniLM-L6-v2` to ONNX then compile to HEF for Pi AI HAT+ NPU)
- `scripts/pi_bio_server.py` (FastAPI server on Pi; accepts bio text, runs HEF via Hailo SDK, returns `bloc_weights`)
- `scripts/generate_synthetic.py` (Gemini 2.5 Pro synthetic training data generation; outputs `data/finetune/synthetic.jsonl`)
- `data/embeddings/` (Parquet cache of raw RoBERTa embeddings per post)
- `data/finetune/train.jsonl`
- `data/finetune/synthetic.jsonl` (weighted 0.5 in regression; Option D)
- `data/finetune/eval.jsonl`
- `data/bio_labels/labeled_bios.jsonl`
- `tests/test_sentiment.py`
- `tests/test_social.py`

- `tests/test_bio_classifier.py`

Optional Overtime (Day 6) — Week 3

Use to ensure the fine-tuning dataset is complete before Week 4's HPC job.

- Process any remaining archives through the cleaning and scoring pipeline
- Submit the Gemini synthetic data generation job and run the MMD weight computation
- Manually inspect 20 fine-tuning examples for obvious errors or format issues
- Write the `train/eval` split script and run it on the full dataset

7 Week 4: LLM Fine-Tuning, Constrained Decoding, and Inference Endpoint

Objective of Week 4

Fine-tune Mistral 7B on the Week 4 dataset, wire constrained decoding, and deliver a live FastAPI inference endpoint. By the end of Week 5:

> *Given a free-text event description, the fine-tuned LLM produces a valid `ShockResponseData` JSON object in under 3 seconds, with schema validity guaranteed by constrained decoding and a quantitative RoBERTa baseline comparison documented.*

CONCEPT: Large language model fine-tuning

A large language model (LLM) is a neural network trained on billions of tokens of text to predict the next token. With enough scale, it internalizes facts, reasoning patterns, and world knowledge. "Fine-tuning" means taking a pre-trained model and continuing to train it on a smaller, task-specific dataset — teaching it new patterns without forgetting what it already knows. For this project, fine-tuning teaches Mistral 7B the relationship between political shock events and demographic loyalty shifts, using 25+ historical examples as the training signal.

Model Selection Rationale

Mistral 7B (`mistralai/Mistral-7B-v0.3`) is selected as the base model:

- **Political and social science knowledge.** Stronger out-of-the-box reasoning on social science tasks than Gemma 3 4B or Llama 3.1 8B, reducing the learning burden on a small fine-tuning dataset.
- **Structured output reliability.** Produces well-formed JSON consistently. Fallback: **Qwen2.5 14B** if structured output proves unreliable in testing (leads structured output benchmarks; fits on M5 in 4-bit).
- **Hardware compatibility.** Runs in full precision on the M5 48GB for local iteration; full training runs pushed to HPC.

- **Ecosystem support.** First-class MLX (Apple Silicon) and HuggingFace PEFT/LoRA support.

> **CONCEPT: Why Mistral 7B specifically**
>
> Open-source models above ∼3B parameters understand complex political and religious language reliably. Mistral 7B is the smallest model that consistently handles nuanced event descriptions ("The Pope endorses a Democratic candidate") without losing semantic coherence. Larger models (13B, 70B) would require GPU memory that CMC's HPC nodes do not provide. Smaller models (3B, 1B) fail on edge cases. Mistral 7B sits at the right point on the capability-cost curve for academic HPC hardware.

Fine-Tuning Approach: QLoRA

Full fine-tuning on ∼500–1,000 examples will overfit. QLoRA fine-tunes adapter parameters $\Delta\theta$ on top of a frozen 4-bit quantized base:

$$h = W_0 x + \Delta W x, \qquad \Delta W = BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll d$$

Parameter	Value
LoRA rank r	16 (try 32 if underfitting)
LoRA α	32
Target modules	Q, K, V, O attention projections
Quantization	4-bit NF4 (`bitsandbytes`)
Learning rate	2×10^{-4}
Epochs	3–5 with early stopping
Held-out set	2020 cycle

Training infrastructure: `mlx-lm` locally on the M5 for fast iteration; PyTorch + HuggingFace `peft` on HPC for full epoch sweeps. Do not run full sweeps locally.

HPC Job Monitoring and Overnight Strategy

The first HPC training run is submitted at the end of Week 5 Day 2 and completes overnight. Do not wait idly:

- **Monitor efficiently.** `squeue -u $USER` checks job status. Once running, `tail -f slurm-$JOBID.out` streams training output. Set up `ssh -L 6006:localhost:6006 hpc` to forward TensorBoard if HPC logs to TensorBoard.
- **Use wait time productively.** While the HPC trains, implement `electoral/llm/eval.py` and `electoral/llm/inference.py` locally on the M5 — these do not require the trained adapter to exist. Use placeholder random weights for unit tests.
- **Prefect idempotency.** The Prefect finetune task checks whether `adapters/mistral-7b-electoral/` exists before submitting. If the adapter is present, the task logs "adapter found, skipping training" and returns immediately. This means re-running the full pipeline after a completed training job costs zero additional HPC time.
- **Auto-copy adapter weights before job exits (critical).** HPC scratch drives are ephemeral — once a compute node is reclaimed by the scheduler, `$SCRATCH` is wiped. **Never rely on a**

manual `scp` after the job finishes. The final lines of `scripts/hpc_submit.sh` must auto-copy the adapter folder to a persistent location before the job exits:

```
# At end of hpc_submit.sh, after training completes:
DEST="$HOME/projects/electoral-equilibrium/adapters"
mkdir -p "$DEST"
cp -r "$SCRATCH/adapters/mistral-7b-electoral" "$DEST/"
echo "Adapter saved to $DEST"
```

Copy to `$HOME/projects/` (persistent across jobs) not `$SCRATCH` (ephemeral). Then Syncthing or `scp` to the M5 as a secondary step once you confirm the copy succeeded.

- **Rank sweep strategy.** Submit rank $r = 16$ first. While it trains, prepare (but do not submit) the $r = 32$ job. Evaluate $r = 16$ on the held-out 2020 set after it completes; only submit $r = 32$ if mean MAE > 0.04. Avoids burning HPC quota on a sweep that may not be needed.

CONCEPT: QLoRA: quantized low-rank adaptation

Full fine-tuning updates all 7 billion parameters of Mistral 7B. This requires ∼112GB of GPU memory in 16-bit precision — far beyond what academic hardware provides. QLoRA solves this with two tricks. First, **quantization**: the frozen base model weights are stored at 4-bit precision (NF4), reducing memory from 112GB to ∼14GB. Second, **low-rank adaptation**: instead of updating the full weight matrices, QLoRA trains two small matrices $A \in \mathbb{R}^{d\times r}$ and $B \in \mathbb{R}^{r\times d}$ for each attention layer, where $r \ll d$ is the rank (typically 16 or 32). The effective weight update is $\Delta W = BA$, trained at full 16-bit precision. Result: ∼95% memory reduction, ∼0.1% of parameters trained, minimal accuracy loss. The adapter weights are saved separately and applied on top of the frozen base model at inference time.

Constrained Decoding — Discrete Bins

LLMs are poor calibrators of precise floating-point values via text generation. Rather than asking Mistral 7B to produce raw $\Delta\mu_i$ floats, the schema uses **nine discrete magnitude bins** that map to numeric distributions post-hoc:

Token	Numeric range
`"strong_neg"`	$[-0.15, -0.09)$
`"mod_neg"`	$[-0.09, -0.05)$
`"mild_neg"`	$[-0.05, -0.02)$
`"weak_neg"`	$[-0.02, -0.005)$
`"neutral"`	$[-0.005, +0.005]$
`"weak_pos"`	$(+0.005, +0.02]$
`"mild_pos"`	$(+0.02, +0.05]$
`"mod_pos"`	$(+0.05, +0.09]$
`"strong_pos"`	$(+0.09, +0.15]$

The LLM outputs a token per bloc (9 possible values each, per race × religion × gender cell). A `bin_to_delta(token)` function maps each token to the midpoint of its range for use in the optimizer.

Do not use hardcoded uniform distributions per bin for uncertainty propagation. The original design used `bin_to_distribution(token)` to return a uniform distribution over the bin's numeric range (e.g., `mild_neg` → Uniform$[-0.05, -0.02)$). This is incorrect: it smears the model's uncertainty uniformly across an *arbitrary* engineering boundary rather than reflecting the true epistemic uncertainty of the prediction. Two events that both output `mild_neg` might have very different empirical variance — one a consistent historical pattern, one a one-off outlier.

Correct approach: empirically-calibrated per-bin uncertainty. During training, for each bin b and each race × religion cell, compute the empirical standard deviation of historical $\Delta\mu$ values that mapped to bin b. Store this as σ_b in `configs/bin_uncertainty.json`. At inference time, `bin_to_distribution(token)` returns $\mathcal{N}(\text{midpoint}(b), \sigma_b^2)$ rather than a uniform. This variance feeds into the Logistic-Normal covariance Σ_Δ for the Monte Carlo rather than producing an artificially hardcoded uncertainty structure. For bins with fewer than 5 historical examples, use a conservative prior $\sigma_b = \text{bin\_width}/2$.

CONCEPT: Constrained decoding

Language models generate text by sampling from a probability distribution over the entire vocabulary (50,000+ tokens) at each step. Nothing prevents the model from generating "the loyalty shift will be approximately negative four point seven percent," which is grammatically plausible but architecturally wrong — we need a structured discrete token. Constrained decoding restricts the sampling distribution at each step to only the tokens that are valid at that position in the output schema. The `outlines` library implements this by taking a JSON schema (the `ShockResponseSchema` Pydantic model) and building a finite-state machine that guides token sampling. The model cannot generate invalid output — the constraint is enforced at the token level, not post-hoc.

WHY THIS DESIGN CHOICE

Why discrete bins rather than continuous floats? Three reasons. First, false precision: the model genuinely does not know whether the shift is −3.7% or −4.2%; a discrete bin like `mild_neg` is more honest. Second, constrained decoding is much simpler with a small fixed vocabulary than with floating-point output. Third, the bins map directly to the optimizer's uncertainty model — each bin corresponds to a distribution of numerical values rather than a point estimate, which feeds into the Logistic-Normal Monte Carlo via empirically-calibrated per-bin standard deviations.

The `outlines` library enforces the 9-token vocabulary per bloc:

```
DELTA_BINS = ["strong_neg","mod_neg","mild_neg","weak_neg","neutral",
              "weak_pos","mild_pos","mod_pos","strong_pos"]

schema = {
    "type": "object",
    "properties": {
        bloc: {"type": "string", "enum": DELTA_BINS}
        for bloc in CANONICAL_BLOCS
    },
```

```
    "required": CANONICAL_BLOCS
}
generator = outlines.generate.json(model, schema)
```

FastAPI Inference Endpoint

```
# electoral/api/shock_endpoint.py
from fastapi import FastAPI
from electoral.llm.inference import estimate_shock

app = FastAPI()

@app.post("/estimate", response_model=ShockResponseData)
async def estimate(event: str, intensity: float = 1.0):
    return estimate_shock(event, intensity)
```

RoBERTa Baseline Comparison

Four methods are evaluated on the held-out 2020 cycle. **The no-information baseline is mandatory for any publication claim** — if `llm_unified` cannot significantly outperform a naive historical average, the structural complexity is unjustified and must be reported honestly.

Method	Input	Metric
`historical_mean`	Historical $\Delta\mu$ averages	MAE per bloc
`roberta_news_only`	News corpus only	MAE per bloc
`roberta_social_only`	Social media only	MAE per bloc
`llm_unified`	Free-text + both scores	MAE per bloc

`historical_mean` **baseline:** for each race × religion × gender cell, predict $\Delta\mu_{i,R}^{(G)} = \bar{\Delta\mu}_{i,R}^{(G)}$ (the historical average shift for that cell across all previous shock events of the same `shock_category`). This is a mean-reverting baseline: it says "this type of event tends to shift this demographic by this amount on average." It requires no model, no NLP, and no data beyond the historical panel. If `llm_unified` cannot significantly outperform it (defined as MAE reduction $> 15\%$ on the held-out 2020 cycle), report both results and discuss in the paper why the more complex model adds value despite modest quantitative gains.

Files to Create

- `electoral/kernels/finetune.py`
- `electoral/llm/trainer.py`
- `electoral/llm/inference.py` (constrained decoding wrapper)
- `electoral/llm/eval.py`
- `electoral/api/shock_endpoint.py`
- `electoral/kernels/shock.py`
- `electoral/models/regression.py` (RoBERTa baseline)

- `adapters/mistral-7b-electoral/`
- `tests/test_llm_output_schema.py`
- `tests/test_constrained_output.py`
- `tests/test_endpoint_latency.py`

Optional Overtime (Day 6) — Week 4

Use if the HPC fine-tuning job needs re-running or the endpoint needs debugging.

- Run the four-way baseline comparison on the held-out 2020 cycle
- Check that `bin_uncertainty.json` per-bin σ_b values are calibrated
- Manually probe the inference endpoint with 10 shock descriptions; check JSON schema
- Document any fine-tuning convergence issues in `DECISIONS.md`

8 Week 5: Stochastic Optimization and Monte Carlo Simulation

Objective of Week 5

Implement the core stochastic optimizer that rebalances voter distributions in response to shocks and runs Monte Carlo simulations to estimate the probability of reaching voter equilibrium. By the end of Week 6:

> *Given a `ShockResponseData` artifact, the system stochastically rebalances the voter distribution using* 10,000+ *Monte Carlo simulations and produces an `EquilibriumData` artifact and a `SimulationData` artifact.*

Connecting the LLM Output to the Optimizer

This is the critical handoff point in the pipeline. The LLM (Week 5) outputs `ShockResponseData.deltas`: a dict of $\Delta\mu_i$ per bloc, representing estimated changes in within-bloc Democratic vote share. The optimizer (this week) needs $\boldsymbol{\mu} + \boldsymbol{\Delta\mu}$ as the new return vector and $\boldsymbol{\Sigma}_\Delta$ as the shock covariance. The connection works as follows:

1. Load `BaselinePortfolioData` to get $\boldsymbol{\mu}$ and $\mathbf{w}^{(0)}$.
2. Load `ShockResponseData.deltas` to get $\boldsymbol{\Delta\mu}$.
3. Compute the post-shock return vector: $\tilde{\boldsymbol{\mu}} = \boldsymbol{\mu} + \boldsymbol{\Delta\mu}$.
4. Estimate $\boldsymbol{\Sigma}_\Delta$ via bootstrap resampling over historical cycles (`bootstrap_cov` in `electoral/models/bootstrap.py`). This captures how uncertain the delta estimates are across different historical analogues — not just the LLM's point estimate.
5. Solve the post-shock QP using $\tilde{\boldsymbol{\mu}}$ and $\boldsymbol{\Sigma}_\Delta$ to produce new coalition weights $\tilde{\mathbf{w}}$.
6. Run Monte Carlo using $\tilde{\boldsymbol{\mu}}$, $\mathbf{w}^*$ (the post-shock optimized weights as the starting point), and $\boldsymbol{\Sigma}_\Delta$ to produce win probability and percentile bands per bloc.

The web app then displays $\tilde{\mathbf{w}}$ (the new coalition weight distribution across the four racial blocs) alongside $\mathbf{w}^{(0)}$ (the baseline), making visible which racial blocs became more or less important after the shock. The chart has two interaction layers: the top-level bars show the four racial blocs (the optimizer's decision variables); clicking any bar expands to show the religious composition within that race (Layer 2), with each religion segment coloured by its post-shock loyalty shift. Gender (Layer 3) is shown as a fill split within each religion segment — female/male vote shares rendered as a vertical fill on the bar — for shocks where gender heterogeneity is significant.

Prediction market prior display. For any event where a live prediction market contract exists (Polymarket or Manifold API query), the web app fetches the current market-implied win probability π^P_{live} and displays it alongside the model's Monte Carlo win probability estimate. The two numbers answer different questions: the market price aggregates all current information about the race; the model output shows how the coalition *structure* must shift given the hypothetical shock. Both are shown with the label "Market consensus: $\pi = \text{X}\%$" and "Coalition model: $P(\text{win}) = \text{Y}\%$" to help users interpret the relationship between them.

CONCEPT: What the optimizer actually does

The optimizer takes the LLM's predicted loyalty shifts and asks: given that this shock has occurred, what is the best way to weight the four racial blocs in the coalition? "Best" means two things simultaneously: maximize the probability of reaching V_{eq} (the win condition), and minimize the volatility of the coalition (minimize variance, so a small perturbation does not flip the outcome). These two objectives trade off against each other, and CVXPY finds the efficient frontier of that trade-off subject to the demographic constraints.

Optimization Formulation

Minimum variance is the wrong objective for deficit scenarios. The Markowitz formulation minimizes coalition variance, which is appropriate when a campaign is ahead and wants to protect a lead. But for campaigns operating from a structural deficit (baseline loyalties below V_{eq}), minimizing variance guarantees a loss — the optimizer flees from exactly the high-variance demographic bets that could produce a tail-event victory. The correct objective is to **maximize the probability of exceeding** V_{eq}, which naturally incorporates the value of variance in deficit scenarios without universally penalizing it.

The re-optimization problem after a shock rebalances the race weights $\mathbf{w}$ while religion weights $\mathbf{v}$ and gender weights $\mathbf{g}$ remain fixed:

$$\max_{\mathbf{w}} \ \Phi\left(\frac{\tilde{\mu}^{\text{eff}}(\mathbf{w}) - V_{\text{eq}}}{\sqrt{\lambda_1^2\, \mathbf{w}^\top \boldsymbol{\Sigma}_\Delta \mathbf{w}}}\right) \quad \text{subject to} \quad \sum_i w_i = 1, \quad w_i \geq 0$$

where $\tilde{\mu}^{\text{eff}}(\mathbf{w}) = \lambda_1 \sum_i w_i(\mu_i^{\text{race}} + \Delta\mu_i^{\text{race}}) + \lambda_2 \sum_R v_R(\mu_R^{\text{rel}} + \Delta\mu_R^{\text{rel}}) + \lambda_3 \sum_G g_G(\mu_G^{\text{gen}} + \Delta\mu_G^{\text{gen}})$, and $\boldsymbol{\Sigma}_\Delta$ is 5×5. This objective is quasi-convex in $\mathbf{w}$ and solved by CVXPY via Disciplined Quasiconvex Programming (DQCP).

DQCP semantics must be declared explicitly — do not rely on automatic solver selection. The Sharpe-ratio-form objective $(\tilde{\mu}^{\text{eff}} - V_{\text{eq}})/\sqrt{\lambda_1^2 \mathbf{w}^\top \Sigma_\Delta \mathbf{w}}$ is a fractional program. It is quasi-convex (each superlevel set is convex), but CVXPY's default solvers (SCS, ECOS,

OSQP) are designed for convex programs and will either fail to converge, return a local non-global optimum, or crash silently on a non-convex formulation.

Use CVXPY's built-in DQCP support: declare the objective with `maximize(cp.ratio(...))`, call `problem.solve(qcp=True)`, and verify that `problem.is_dcp(qcp=True)` returns `True` before solving. DQCP uses bisection to reduce the quasiconvex problem to a sequence of convex feasibility problems, guaranteeing the global optimum. Add a test: `assert problem.is_dcp(qcp=True)` must pass before any SLURM fine-tuning job is submitted.

The feasibility flag in `EquilibriumData` fires when no $\mathbf{w}$ on the simplex can push $\tilde{\mu}^{\text{eff}}$ above V_{eq}.

CONCEPT: Why Monte Carlo?

The optimizer returns a single optimal weight vector $\mathbf{w}^*$ — a point estimate. But the win probability depends on the uncertainty in future loyalty outcomes, not just the point estimate. Monte Carlo simulation samples thousands of plausible coalition scenarios near $\mathbf{w}^*$ and computes what fraction of them satisfy the win condition. This gives a *distribution* over win probabilities — a 95% confidence interval, not just a single number. A coalition with 51% win probability and low variance is more reliable than 51% with high variance; the gauge in the app reflects this distinction.

CONCEPT: Dirichlet distribution

A Dirichlet distribution is a probability distribution over probability vectors — vectors that sum to 1. It is the natural distribution for coalition weights because it automatically enforces the simplex constraint ($\sum w_i = 1$, $w_i \geq 0$) without any projection step. Parameterized by $\boldsymbol{\alpha}$: when all α_i are large, samples cluster tightly near $\alpha_i / \sum_j \alpha_j$ (low uncertainty); when α_i are small, samples spread across the simplex (high uncertainty). We set $\boldsymbol{\alpha}$ using method-of-moments from $\mathbf{w}^*$ and the diagonal of Σ_Δ, which converts the optimizer's point estimate and historical variance into Dirichlet concentration parameters.

WATCH OUT

The naive approach — drawing $\eta \sim \mathcal{N}(\mathbf{0}, \Sigma_\Delta)$, adding to $\mathbf{w}^*$, then projecting onto the simplex — is *wrong*. Simplex projection truncates negative components asymmetrically, destroying the Gaussian covariance structure. The resulting distribution is no longer the intended one. Use Dirichlet sampling directly: it lives on the simplex by construction with no projection needed.

Monte Carlo Simulation

CONCEPT: Why the Dirichlet distribution is wrong here

A Dirichlet distribution's off-diagonal covariances are *always* negative: $\text{Cov}(w_i, w_j) = -\alpha_i \alpha_j / (\alpha_0^2(\alpha_0 + 1)) < 0$ for all $i \neq j$. This means Dirichlet Monte Carlo cannot model a scenario where two blocs move in the same direction simultaneously — but wave elections are defined precisely by correlated shifts across blocs (a macroeconomic shock suppresses Latino Catholic *and* White working-class loyalty simultaneously). The Dirichlet would force them

to move inversely, generating fictitious coalition dynamics with no basis in electoral history. Additionally, a K-dimensional Dirichlet has only K degrees of freedom, while a full covariance matrix has $K(K+1)/2$ — it is impossible to embed an arbitrary covariance structure into a Dirichlet.

Sampling method: Logistic-Normal distribution. The correct distribution for simplex sampling with arbitrary covariance is the **Logistic-Normal** (Aitchison & Shen, 1980). It operates in unconstrained space and maps onto the simplex via the additive logistic transformation.

The delta method floor ($w_i^* \geq 0.01$) introduces non-linear distortion in Aitchison geometry — replace with ILR parameterization.

The delta method step $\Sigma_{\ln,ij} \approx \Sigma_{\Delta,ij}/(w_i^* w_j^*)$ explodes when any $w_i^* \to 0$. The 0.01 floor prevents numerical catastrophe but introduces a *non-linear* distortion in log-ratio space: what appears as a tiny 1% perturbation in probability space corresponds to a massive shift in the logarithmic geometry of the simplex near its boundaries. The resulting CI bands reflect the floor value, not the true covariance.

Correct approach: isometric log-ratio (ILR) parameterization. Rather than transforming Σ_Δ via the delta method, parameterize the Logistic-Normal directly in the ILR coordinate system:

1. Map $\mathbf{w}^*$ to ILR coordinates: $\mathbf{z}^* = V^\top \log(\mathbf{w}^*)$, where V is any $(K-1) \times K$ contrast matrix satisfying $VV^\top = I_{K-1}$. A standard choice is the Helmert contrast matrix.
2. Propagate Σ_Δ to ILR space via the delta method with respect to the ILR transformation (not the raw log): the Jacobian is better conditioned because ILR coordinates are orthonormal and bounded away from singularity at interior points.
3. Draw $\mathbf{y}^{(n)} \sim \mathcal{N}(\mathbf{z}^*, \Sigma_{\text{ILR}})$ in $\mathbb{R}^{K-1}$.
4. Back-transform: $\mathbf{w}^{(n)} = \text{clr}^{-1}(V\mathbf{y}^{(n)})$ via the softmax map.

If the optimizer returns $w_i^* = 0$ exactly for some bloc, treat this as a boundary case and report it as `infeasible_bloc` in `EquilibriumData` rather than applying an arbitrary floor. The Monte Carlo is run only on the interior blocs with $w_i^* > 0$. Blocs assigned zero weight by the optimizer are excluded from the simulation — they are genuinely not in the optimal coalition. This is architecturally honest; the floor was masking a real optimization result.

1. Map $\mathbf{w}^*$ to ILR coordinates $\mathbf{z}^* = V^\top \log(\mathbf{w}^*)$ using a Helmert contrast matrix V.
2. Propagate $\mathbf{\Sigma}_\Delta$ to ILR space: $\Sigma_{\text{ILR}} = J\Sigma_\Delta J^\top$ where J is the Jacobian of the ILR transformation evaluated at $\mathbf{w}^*$.
3. Draw $N = 10{,}000+$ samples $\mathbf{y}^{(n)} \sim \mathcal{N}(\mathbf{z}^*, \Sigma_{\text{ILR}})$ in $\mathbb{R}^{K-1}$.
4. Back-transform via softmax: $w_i^{(n)} = \exp(y_i^{(n)}) / \sum_j \exp(y_j^{(n)})$.
5. Record whether $\tilde{\mu}^{\text{eff}}(\mathbf{w}^{(n)}) \geq V_{\text{eq}}$ for each sample.
6. Compute win probability and 90% CI bounds across the N samples.

This preserves the full off-diagonal covariance structure of $\mathbf{\Sigma}_\Delta$ — including positive correlations be-

tween blocs that move together under macroeconomic or geopolitical shocks — while guaranteeing all samples lie on the probability simplex. Implement in `electoral/simulation/montecarlo.py` using `scipy.stats.multivariate_normal` for the draw and `scipy.special.softmax` for the projection.

Win-probability confidence interval (required output). In addition to the point-estimate win probability, compute the 90% confidence interval across Monte Carlo samples:

- **Point estimate:** fraction of N samples where $\mathbf{w}^{(n)\top}\tilde{\mu}^{\text{eff}} \geq V_{\text{eq}}$
- **Lower bound (p=0.05):** 5th percentile of the per-sample win-probability distribution
- **Upper bound (p=0.95):** 95th percentile

The WinGauge displays the 90% CI as a shaded band around the point estimate. The "No feasible path" banner fires only when the *upper* CI bound cannot reach V_{eq} — meaning even optimistic coalition scenarios fail. If the lower bound is below V_{eq} but the upper bound is above it, the banner reads "Uncertain path" rather than a hard failure. Store in `SimulationData` as `win_probability_low` and `win_probability_high`.

Near-degenerate coalition test (required). Add `test_montecarlo_degenerate`: instantiate with $w_0^* = 0.99$, $w_{1..3}^* = 0.003\overline{3}$; assert all N samples sum to 1.0 within `tol=1e-9` and `win_probability` is in $[0, 1]$.

Methodological limitations (must be discussed in the paper).

Small-n covariance instability. The covariance matrix $\mathbf{\Sigma}_\Delta$ is estimated from 6–8 historical election cycles. At this sample size, individual outlier cycles (2008 financial crisis, 2020 pandemic) dominate the estimate. LedoitWolf shrinkage guarantees positive-definiteness and satisfies CVXPY's preconditions, but cannot conjure reliable signal from six observations. The confidence intervals on Monte Carlo win probability will be wide; report them honestly. Partially mitigate by weighting recent cycles (2012–2024) more heavily via exponential decay in the covariance estimator.

Turnout–preference endogeneity. The optimizer treats coalition weights w_i (turnout allocation) as a flexible decision variable and loyalty μ_i (preference) as an exogenous LLM output. In reality, preference shocks routinely depress turnout in the same direction — a shock that suppresses Latino loyalty also tends to suppress Latino turnout. The model does not capture this co-movement. Report this as a limitation: the optimizer identifies the mathematically optimal coalition structure, but achieving it requires independent campaign efforts on turnout that the model does not model.

Sparse cell imputation bias. Small race × religion intersections (Asian Muslim, Latino Jewish, Asian Jewish) have limited or no survey data in ARDA/GSS for most historical cycles. These cells are imputed from group averages, which biases their effective loyalty toward majority-group estimates. The optimizer's minimum-variance objective will then systematically overweight the White racial bloc (densest data, lowest imputation error) regardless of strategic value. Bound the White coalition weight to $w_{\text{white}} \leq 0.80$ to prevent this artifact from dominating the output.

Files to Create

- `electoral/kernels/optimize.py`
- `electoral/optimization/cvx.py` (CVXPY formulations)
- `electoral/optimization/simplex.py` (simplex projection)
- `electoral/models/bootstrap.py` (covariance estimation via resampling over historical cycles)
- `electoral/simulation/montecarlo.py`
- `tests/test_optimization.py`
- `tests/test_montecarlo.py`
- `tests/test_bootstrap.py`

Optional Overtime (Day 6) — Week 5

Use to stress-test the optimizer and Monte Carlo before wiring the frontend.

- Run the optimizer across all 25+ shock events; verify no degenerate solutions
- Check that the essential-zeros floor ($w_i \geq 0.01$) prevents log-ratio explosion
- Verify the CI band (90% interval) is plausibly wide given the covariance estimate
- Profile the end-to-end latency (LLM + optimizer + Monte Carlo) and document in `DECISIONS.md`

9 Week 6: Web Application

Objective of Week 6

Wire the Next.js frontend to the FastAPI backend and deliver a working end-to-end demo. By the end of Week 7:

> *A user selects a party, enters a free-text hypothetical shock event, and sees two things rendered in real time: (1) what the model predicts will happen — a plain-language narrative and per-bloc loyalty shift chart showing how each voter group is expected to move; and (2) the most likely winning rebalance — the optimizer's recommended new coalition weights with the probability of achieving them, displayed alongside the baseline so the user can see exactly which groups must grow or shrink in importance.*

CONCEPT: Server-Sent Events (SSE)

SSE is a web protocol for one-way streaming from server to browser over a single persistent HTTP connection. The browser opens the connection with `new EventSource(url)` and registers handlers for named events. The server pushes `data: ...\n\nevent: name` strings whenever results are ready. Unlike WebSockets, SSE is unidirectional (server to client only), simpler to implement, and works through standard HTTP/2 multiplexing. For this project,

SSE is ideal: the user submits one request and then just waits for three events (`deltas`, `equilibrium`, `simulation`) to arrive sequentially.

CONCEPT: Modal serverless GPU

Modal is a cloud platform that runs Python functions on serverless GPU instances. You decorate a function with `@app.function(gpu="A100")` and Modal handles provisioning, scaling, and teardown. Cold start takes ∼5 seconds; warm instances respond in ∼200ms. For this project: the LLM inference function runs on a Modal A100 GPU; the optimizer runs on Modal CPU. This eliminates the need to manage a persistent GPU server, which would cost $800+/month. Modal's $30/month free credit covers development entirely at ∼$0.0003 per inference call.

CONCEPT: Next.js and Vercel

Next.js is a React framework that handles routing, server-side rendering, and API routes. Vercel is its hosting platform — you push to GitHub and Vercel deploys automatically via CDN in ∼30 seconds. The frontend is a static React application (no server-side logic) that makes one POST request to the Modal backend and then listens on an SSE stream. Vercel's free hobby tier covers this with no cost.

CONCEPT: DuckDB

DuckDB is an embedded analytical database that runs inside the Python process with no separate server. It is optimised for analytical workloads (aggregations, window functions, GROUP BY) rather than high-throughput OLTP writes. The analyst dashboard runs queries like "show me all inference calls from the past 30 days where win_probability > 0.6 and feasible = false" — exactly the analytical pattern DuckDB handles well.

WATCH OUT

Do not use DuckDB for live serverless write appends. The asyncio.Queue single-writer pattern prevents intra-process lock contention, but Modal's serverless infrastructure scales horizontally by spinning up *multiple isolated container instances* under load. Multiple containers trying to write to the same embedded DuckDB file via shared network storage will trigger file-lock exceptions and drop audit records.

Two-tier audit architecture:

1. **Live write sink:** Each Modal container appends audit records to a decoupled **PostgreSQL instance** (Supabase free tier: 500MB, sufficient for thousands of inference calls). FastAPI writes one row per inference call. No shared file; no lock contention across containers.

2. **Local analytical layer:** The analyst dashboard on the M5 queries PostgreSQL directly, or periodically exports to a local DuckDB file for offline analytical queries. DuckDB is **read-only** in the dashboard context — it never receives live writes from Modal.

This keeps DuckDB's analytical strengths for the dashboard while eliminating the multi-container write fragility in the serverless inference path.

WATCH OUT

CVXPY's C solvers are not thread-safe. If two simultaneous user requests both trigger the optimizer inside an async FastAPI endpoint using the default thread pool, their solver states will corrupt each other and crash. Fix: run CVXPY in a `ProcessPoolExecutor(max_workers=1)` — a separate process with isolated memory. One optimization runs at a time. The Monte Carlo (pure NumPy) is thread-safe and runs in a `ThreadPoolExecutor`.

Web Application Architecture

The public app answers one question in two parts. Given a shock event and a party:

1. **What will happen?** The LLM estimates how each of the 10 voter blocs shifts in loyalty toward the selected party after the shock. This is the *prediction* — grounded in historical analogues and social media signal.

2. **What must change to still win?** The optimizer finds the minimum-variance rebalancing of coalition weights that still satisfies the win threshold under the shifted loyalty estimates. This is the *prescription* — the most likely path to maintaining a winning coalition, with a probability attached.

Both outputs appear in the same view, separated visually into two panels. The user does not need to navigate between them.

- **Frontend:** Next.js with Recharts and Radix UI. Five core components: `ShockInput` (party toggle + event text + intensity slider), `ShockNarrative` (plain-language prediction summary), `CoalitionChart` (single unified bar chart: opaque bars for the prediction, translucent bars for the rebalance, both on the same axis), and `WinGauge` (win probability semicircle + market prior). Results stream progressively via SSE: LLM deltas arrive first and immediately populate `ShockNarrative` and the opaque bars of `CoalitionChart`; optimizer weights arrive next and add the translucent rebalance layer in-place; Monte Carlo win probability arrives last and fills `WinGauge`.

- **Panel layout.** `ShockNarrative` sits above `CoalitionChart`, full width. `WinGauge` sits to the right of the chart on desktop, below on mobile.

- **CoalitionChart design — two layers, one chart.** The chart renders 10 horizontal bars (one per bloc), sorted by magnitude of loyalty shift descending. Each bloc row carries two overlapping bars:

  - **Opaque** (`fillOpacity=1.0`): post-shock loyalty $\tilde{\mu}_i$ — the model's *prediction* of how each group's support shifts after the event.

  - **Translucent** (`fillOpacity=0.35`): optimizer coalition weight $\tilde{w}_i$ — the minimum-variance *rebalance* that still achieves the win threshold.

  A thin gray reference tick marks the pre-shock baseline μ_i. A color-coded legend in the chart header labels each layer: "— Prediction" (solid) and "— Rebalance" (translucent). Both series

use the same party color (blue for Democrat, red for Republican); opacity alone distinguishes them. If `feasible=false`, the translucent layer is replaced with a striped red pattern and a "No feasible path" banner.

The translucent layer is absent until the `event: equilibrium` SSE event fires. The opaque bars are fully readable while the optimizer runs — the user sees the prediction immediately and the rebalance overlays a moment later.

- **Panel 1 — What will happen (Prediction).** `ShockNarrative` renders a plain-language summary from the LLM delta bins. Below it, `CoalitionChart` immediately renders the opaque bars (post-shock loyalty $\tilde{\mu}_i$) the moment the `event: deltas` SSE event fires. The user can read the prediction before the optimizer finishes.

- **Panel 2 — Most likely rebalance (Prescription).** When `event: equilibrium` fires, `CoalitionChart` overlays the translucent bars ($\tilde{w}_i$, 35% opacity) on top of the existing opaque bars in-place. No re-render, no flicker. `WinGauge` populates when `event: simulation` fires. If `feasible=false`, the translucent layer is replaced with a striped red pattern and a "No feasible path" banner.

- **Backend:** FastAPI with two response modes: `POST /estimate` for blocking JSON (used by automated tests) and `GET /estimate/stream` for SSE streaming (used by the web app). The SSE endpoint emits three events in sequence: `deltas` (LLM output → populates narrative + opaque bars), `equilibrium` (optimizer output → adds translucent layer), `simulation` (Monte Carlo → populates WinGauge).

  Thread safety: CVXPY's underlying C solvers (SCS, ECOS, OSQP) are not thread-safe. Use `ProcessPoolExecutor` for the optimizer and `ThreadPoolExecutor` for Monte Carlo (pure NumPy, GIL-releasing).

- **Analyst dashboard (Palantir-aligned):** A password-protected `/dashboard` route showing pipeline internals: data coverage matrix (bloc × cycle), RoBERTa score distributions, bio classifier coverage rates, fine-tuning loss curves, Monte Carlo convergence diagnostics, and a DuckDB query interface over the audit log. Built with the same Next.js frontend; accessible only with a `DASHBOARD_PASSWORD` environment variable set.

- **Audit logging (Palantir-aligned):** Every `/estimate` call persists to a DuckDB database: timestamp, input event text, intensity, LLM deltas per bloc, optimizer feasibility, win probability, latency per stage. The dashboard renders this as a queryable table. Every inference call is traceable end-to-end.

- **Hosting:** Modal (primary) + Vercel (frontend). HPC vLLM scaffolded as one-env-var alternative for post-SRP activation.

Analyst Dashboard

The analyst dashboard is a password-protected internal tool built on top of the same Next.js frontend, accessible at `/dashboard`. It is architecturally distinct from the public-facing shock simulator: where the public app answers "what happens if X occurs?", the dashboard answers "how well is the pipeline working and what has it been asked?" This distinction — end-user tool vs. analyst tool — is the core of Palantir's Foundry product model and is intentionally replicated here.

Authentication

The dashboard checks a `DASHBOARD_PASSWORD` environment variable against a session cookie set on login. No external auth service is required. The cookie expires after 8 hours. The `/api/dashboard/auth` route handles login and logout.

Dashboard Panels

Panel 1 — Data Coverage Matrix. A Recharts heatmap (`bloc_id` × `cycle`) showing the data availability of the voter panel. Each cell is colored by data quality: dark blue = multiple sources agree, light blue = single source, gray = imputed, white = missing. Sourced from the `VoterPanelData` artifact. Clicking a cell opens a tooltip showing the source breakdown and raw `vote_share` value. This panel directly addresses the reviewer question "how complete is your dataset?"

Panel 2 — RoBERTa Score Distributions. A per-bloc violin plot showing the distribution of baseline-adjusted sentiment elasticity scores $\Delta\varepsilon_{i,s}$ across all 25+ shock events. Sourced from the `SentimentData` artifact. High variance in a bloc's distribution signals that bloc is shock-sensitive; low variance signals consistency. A shock category filter (Security, Geopolitical, Moral, Electoral Surprise) allows slicing by category. This is the key diagnostic for whether the social media pipeline is producing meaningful signal.

Panel 3 — Bio Classifier Coverage. A grouped bar chart showing per-shock-event bio classifier coverage: for each of the 25+ shocks, the fraction of X posts that were (i) keyword-matched, (ii) SetFit-classified, or (iii) fell back to the platform proxy. Sourced from coverage metadata written to `data/embeddings/` by the scorer. Low coverage (many fallbacks) on a specific shock signals the keyword lexicon may need expanding for that event type.

Panel 4 — Fine-Tuning Loss Curves. Training and validation loss curves for the Mistral 7B QLoRA run, loaded from the HPC training logs. Shows convergence behavior and whether the model overfit the training set. The held-out 2020 cycle MAE is displayed as a horizontal reference line. A "training run selector" dropdown allows comparing multiple HPC runs (rank 16 vs. rank 32, real-only vs. real+synthetic).

Panel 5 — Monte Carlo Convergence. Win probability vs. N for the most recent shock estimate, from `SimulationData.percentiles`. Confirms the simulation has converged before reporting results. The plot shows p5, p50, and p95 bands narrowing as N increases; the convergence gate (± 0.005 between $N = 5{,}000$ and $N = 10{,}000$) is shown as a shaded tolerance band.

Panel 6 — Audit Log Query Interface. A sortable, filterable table of all `/estimate` calls logged to `data/audit.duckdb`, served by `GET /api/audit`. Columns: timestamp, event text (truncated), intensity, win probability, feasibility, latency per stage. A free-text filter box runs a SQL `LIKE` query over the event text. Clicking any row expands a detail view showing the full per-bloc delta vector. This makes every inference call fully traceable and queryable.

Backend API Routes for the Dashboard

Route	Returns	Source
`GET /api/dashboard/auth`	Session cookie	`DASHBOARD_PASSWORD` env
`GET /api/coverage`	Panel 1 coverage matrix	`VoterPanelData` artifact
`GET /api/sentiment-dist`	Panel 2 score distributions	`SentimentData` artifact
`GET /api/bio-coverage`	Panel 3 bio classifier stats	`data/embeddings/` metadata
`GET /api/training-logs`	Panel 4 loss curves	HPC log files (via Syncthing)
`GET /api/convergence`	Panel 5 MC convergence	`SimulationData` artifact
`GET /api/audit`	Panel 6 estimate log	`data/audit.duckdb`

DuckDB Write Concurrency

DuckDB is an embedded database. If multiple concurrent FastAPI coroutines attempt to write to `data/audit.duckdb` simultaneously — which happens immediately under any real traffic since FastAPI is async — the file will be locked and writes will fail with an I/O error. The single-machine design prevents cross-process contention, but within-process concurrency from async request handlers is a different problem.

Solution: `asyncio.Queue` with a single background writer. `audit.py` implements a singleton `AuditLogger` with:

- An `asyncio.Queue` that any coroutine can `put_nowait()` an `AuditEntry` dataclass into without blocking.
- A single background `asyncio.Task` (started at FastAPI startup via `@app.on_event("startup")`) that `get()`s entries from the queue and executes the DuckDB `INSERT` — the only coroutine that ever touches the `.duckdb` file for writes.
- Read queries (`GET /api/audit`) open DuckDB in read-only mode (`duckdb.connect(read_only=True)`), which does not conflict with the single writer.

This pattern makes audit logging non-blocking from the request handler's perspective and guarantees serialised writes without a lock manager.

Audit Log Schema

The `estimates` table in `data/audit.duckdb`:

```
CREATE TABLE estimates (
    id              INTEGER PRIMARY KEY,
    timestamp       TIMESTAMP,
    event_text      VARCHAR,
    party           VARCHAR,     -- "democrat" or "republican"
    intensity       FLOAT,
    -- Per-bloc LLM delta estimates (stored as JSON string)
    deltas_json     VARCHAR,
    -- Optimizer output
    feasible        BOOLEAN,
    target_met      BOOLEAN,
```

```
    win_prob       FLOAT,
    -- Per-stage latency (ms)
    llm_ms         INTEGER,
    optimizer_ms   INTEGER,
    montecarlo_ms  INTEGER,
    -- Source tracking
    backend        VARCHAR   -- "modal" or "hpc"
);
```

UI Uncertainty Communication

The web app must display uncertainty prominently. The win-probability gauge shows a point estimate but the percentile distribution strip below it shows the p5–p95 range across Monte Carlo simulations. A disclaimer is shown beneath every result: *"Estimates carry significant uncertainty. This tool is a research prototype and should not be used for campaign strategy or electoral decisions."*

Files to Create

- `webapp/app/page.tsx` (main layout with SSE `EventSource`; progressive reveal: narrative → opaque bars → translucent layer → gauge)
- `webapp/app/dashboard/page.tsx` (analyst dashboard, password-protected)
- `webapp/components/ShockInput.tsx` (party toggle + event textarea + intensity slider)
- `webapp/components/ShockNarrative.tsx` (plain-language prediction summary from LLM delta bins; populates immediately on `event: deltas`)
- `webapp/components/CoalitionChart.tsx` (unified bar chart: opaque bars for prediction $\tilde{\mu}_i$, translucent overlay for rebalance $\tilde{w}_i$; feasibility banner if optimizer finds no solution)
- `webapp/components/WinGauge.tsx` (win probability semicircle + market prior; populates on `event: simulation`)
- `webapp/components/AuditLog.tsx` (queryable DuckDB table view)
- `webapp/components/CoverageMatrix.tsx` (bloc × cycle data coverage)
- `webapp/lib/api.ts` (typed FastAPI client; SSE and blocking modes)
- `webapp/lib/types.ts` (TypeScript mirrors of backend payload schemas)
- `electoral/api/shock_endpoint.py` (adds `/estimate/stream` SSE route)
- `electoral/api/audit.py` (DuckDB audit log writer and reader)
- `data/audit.duckdb` (persistent audit log database)
- `deploy/modal_app.py`
- `deploy/hpc_vllm.sh`
- `deploy/backend_router.py`

- `vercel.json`
- `scripts/start_demo.sh`

Optional Overtime (Day 6) — Week 6

Use to pre-stage hardening tasks or get a head start on the paper.

- Run the full integration test suite end-to-end and document failures
- Begin the methodology section of the paper draft
- Test the app on mobile; fix any layout issues in the CoalitionChart
- Smoke-test the replication package on a clean checkout

10 Week 7: Hardening, Validation, and Deliverables

Objective of Week 7

Harden the pipeline, pass all validation gates, and finalize the three primary deliverables. By the end of Week 8:

> *The pipeline produces deterministic artifacts across repeated runs, the full test suite passes, and the research paper draft, technical poster, and deployed web application are complete.*

Required Tests

- **End-to-end integration test.** Run the pipeline on the smoke configuration; assert all stage artifacts exist and parse correctly.
- **Full-run determinism test.** Run the pipeline twice with the same config and seed; assert content-identical `data` payloads for each stage artifact.
- **Artifact schema gate.** Load each artifact and assert envelope keys and payload fields match their declared schemas.
- **Monte Carlo convergence test.** Assert that win probability estimates converge to within ± 0.005 between $N = 5{,}000$ and $N = 10{,}000$ simulations.
- **Constrained output test.** Assert that 100 consecutive LLM inference calls all produce schema-valid `ShockResponseData` JSON.
- **Endpoint latency test.** Assert the FastAPI `/estimate` endpoint responds in under 3 seconds on the M5.

Deliverables

1. **Formal Research Paper.** Co-authored manuscript for submission to an interdisciplinary journal (e.g., *Political Behavior*, *Journal of Elections, Public Opinion and Parties*). Sections:

Introduction, Literature Review, Methodology (two-stage RoBERTa–LLM pipeline as methodological contribution), Results, Discussion, Appendix (replication instructions).

Required in the methodology section:

- **Quasiconvexity proof.** A formal demonstration that the Sharpe-ratio-form objective $f(\mathbf{w}) = (\tilde{\mu}^{\text{eff}}(\mathbf{w}) - V_{\text{eq}})/\sqrt{\lambda_1^2 \mathbf{w}^\top \Sigma_\Delta \mathbf{w}}$ is quasi-convex in $\mathbf{w}$ over the simplex constraint set. The proof outline: (1) the numerator is linear (hence convex and concave) in $\mathbf{w}$; (2) the denominator $\sqrt{\mathbf{w}^\top \Sigma_\Delta \mathbf{w}}$ is convex in $\mathbf{w}$ for positive-definite Σ_Δ; (3) the ratio of a linear function to a positive convex function is quasi-convex on the domain where the denominator is strictly positive (Diamond & Boyd, DQCP 2019). Since Σ_Δ is positive-definite by LedoitWolf regularization and all simplex weights are non-negative, the denominator is strictly positive for any $\mathbf{w} \neq 0$. Therefore f is quasi-convex and CVXPY's DQCP solver returns the global optimum. Include this proof verbatim or cite the DQCP paper (Diamond et al., 2019, arXiv:1905.00562).

- **Additive stratum approximation statement.** Explicitly state that the parallel stratum effective loyalty formula assumes additive independence across race, religion, and gender, and that this is a testable approximation. Report the in-sample fit on held-out cycles.

- **Raking comparison.** Report both the additive model output and the raking-calibrated output. Discuss any divergence.

2. **Technical Conference Poster.** Conference-style poster ($48'' \times 36''$) presenting model architecture, the RoBERTa vs. LLM comparison, and win-probability distributions under representative shock scenarios. Live web app demo embedded via QR code.

3. **Interactive Web Application.** Public-facing Next.js app with FastAPI backend serving the fine-tuned Mistral 7B via constrained decoding. Users select a party (Democrat or Republican), input a free-text hypothetical event and intensity level; the app displays how the distribution of the 10 key voter groups shifts under the shock for that party's coalition, alongside the re-equilibrated coalition weights and estimated win probability $P(\tilde{\mathbf{w}}^\top \tilde{\boldsymbol{\mu}}^{(P)} \geq V_{\text{eq}}^{(P)})$. Results stream progressively via SSE. Deployed on Modal + Vercel.

4. **Generalization Scaffolding.** Every domain-specific parameter is named in `configs/base.json` rather than hardcoded. Every core module (optimizer, Monte Carlo, scorer, LLM estimator) is written against abstract interfaces with the electoral vertical as the concrete implementation. `DOMAIN.md` documents the full specification for instantiating a new domain. This positions the codebase as the technical foundation for future verticals (shareholder coalition management, legislative modeling, monetary policy) without requiring architectural refactoring.

   Password-protected internal tool at `/dashboard` showing six panels: data coverage matrix, RoBERTa score distributions, bio classifier coverage, fine-tuning loss curves, Monte Carlo convergence diagnostics, and a queryable DuckDB audit log of every inference call. This is the Palantir-aligned deliverable — it demonstrates the distinction between an end-user tool and an analyst tool, which is the core of Palantir's Foundry product model. Include a screenshot of the dashboard in the conference poster.

5. **Continuous Update Architecture.** The model must not become stale after the SRP ends. The pipeline is designed with three layers of ongoing evolution:

   (a) **Continuous data ingestion.** A nightly launchd job on the **Intel Mac** (already running collectors 24/7) triggers a Prefect flow at 2am in `pipeline_mode="continuous"`: runs `BlueSkyCollector` and `ApifyCollector` for any shock events that occurred in the last 7 days, scores the new posts via RoBERTa, and appends to `data/finetune/new_events.jsonl`. When the fine-tuning threshold is reached, the Intel Mac submits a SLURM job to the HPC rather than training locally. The voter panel data (ARDA, GSS, Pew) is version-checked annually — when new data is released, re-run the Week 2 pipeline.

   (b) **Periodic fine-tuning.** When `new_events.jsonl` accumulates 10+ new shock events, a new fine-tuning run is triggered on the HPC. The new adapter replaces the old one (old adapter archived with timestamp). The win threshold $V_{\text{eq}}^{(P)}$ and constraint bounds update automatically each cycle because `build_constraint_spec` derives them from the voter panel data rather than hardcoding them.

   (c) **Prediction market calibration.** Every inference call is logged to `audit.duckdb`. After each shock event resolves, the model's `win_probability` can be compared against the actual market outcome. Systematic divergence (e.g. consistently over-estimating Evangelical sensitivity to economic shocks) surfaces in the audit log and informs recalibration of the fine-tuning example weights (market-backed, poll-only, synthetic). This feedback loop is the long-term moat: the model improves the more it is used.

   The handoff document `CONTINUOUS_UPDATE.md` specifies the scheduled pipeline, the fine-tuning trigger threshold, and the calibration procedure, so the model can be maintained after the SRP ends.

Replication Package Checklist

- `README.md` — "How to Reproduce This Run" instructions
- `configs/paper_baseline.json` — frozen configuration for manuscript results
- `artifacts/` — all stage artifacts for the paper baseline run
- `adapters/mistral-7b-electoral/` — frozen QLoRA adapter weights
- `pyproject.toml` — pinned dependency versions
- `AGENTS.md` — contributor conventions and PR standards
- `CONTINUOUS_UPDATE.md` — scheduled pipeline spec, fine-tuning trigger threshold, and calibration procedure for post-SRP model maintenance

Files to Create or Modify

- `tests/test_integration_end_to_end.py`
- `tests/test_determinism_full_run.py`
- `tests/test_artifact_schemas.py`

- `tests/test_mc_convergence.py`

11 Expected Outcome at Week 8

- Produce deterministic artifacts for every pipeline stage.
- Support multiple portfolio rebalancing methods (equal-weight baseline, GMV re-optimization, shock-constrained stochastic rebalancing).
- Generate manuscript-ready performance tables and win-probability distributions.
- Pass integration and full-run determinism tests.
- Be readable and extensible for future election cycles (2028 and beyond).
- Serve as a publicly accessible analytical tool bridging academic research and civic data literacy.

A AI Research Assistant Tooling

Overview

This project uses a self-hosted AI agent stack to support research tasks throughout the 7-week period. The stack runs on a Raspberry Pi 5 acting as a 24/7 gateway, routing queries through OpenClaw to Google Gemini Pro for routine tasks and to Claude (via the Anthropic API) for deep architectural and reasoning work. This keeps Claude Max usage reserved for serious development sessions while offloading lightweight research queries — paper summaries, quick code questions, draft text — to Gemini at near-zero cost.

Stack Components

Component	Role	Hardware
OpenClaw (open-source agent gateway)	Orchestration, memory, scheduling	Raspberry Pi 5
Google Gemini Pro (via AI Studio API)	Lightweight daily research queries	Cloud (API)
Anthropic Claude (via API or Max sub)	Deep reasoning, architecture, code	Cloud (API)
Telegram / WhatsApp	Primary chat interface	Phone

Why This Split

OpenClaw is model-agnostic: it routes each request to whichever provider is configured for that agent. The routing strategy for this project is:

- **Gemini Pro** handles: paper summarization, quick factual lookups, draft text generation, calendar and email management, daily briefings. Gemini's free tier (60 requests/minute, 1,000/day via Google AI Studio) covers nearly all routine queries at no cost. The Gemini Pro subscription

provides higher quota and access to the full Gemini 3.x model family with its 1M-token context window — useful for ingesting long papers or full codebases in a single query.

- **Claude** handles: architectural decisions, debugging complex pipeline logic, writing or reviewing LaTeX, designing experiments, and any task requiring sustained multi-step reasoning over the full project context. Claude Max usage is preserved for development sessions, not background tasks.

Setup Instructions

1. Install OpenClaw on the Raspberry Pi 5

```
# On the Pi (SSH in first)
curl -fsSL https://openclaw.ai/install.sh | bash
openclaw onboard
```

During `openclaw onboard`, select Telegram or WhatsApp as the primary channel. Have your Google AI Studio API key and Anthropic API key ready.

2. Configure Gemini as the default model provider

Add the following to your OpenClaw config (open with `openclaw config edit`):

```
{
  "models": {
    "providers": {
      "google": {
        "apiKey": "$GEMINI_API_KEY",
        "models": ["google/gemini-2.5-pro", "google/gemini-2.5-flash"]
      },
      "anthropic": {
        "apiKey": "$ANTHROPIC_API_KEY",
        "models": ["claude-sonnet-4-6"]
      }
    },
    "defaults": {
      "chat": "google/gemini-2.5-pro",
      "fast":  "google/gemini-2.5-flash"
    }
  }
}
```

Set both keys as environment variables in `/etc/environment` on the Pi so they persist across reboots. Never commit API keys to the repository.

3. Configure OpenClaw skills for research tasks

Install the following skills from ClawHub to cover the most common research assistant workflows:

```
openclaw skills install paper-summarizer   # summarize PDFs / arXiv URLs
openclaw skills install code-reviewer      # review Python files from codebase
openclaw skills install morning-brief      # daily summary: calendar + GitHub
openclaw skills install web-search         # search and summarize URLs
```

4. Switch to Claude for deep work

When a task requires sustained reasoning, prefix your message with `@claude` to route that query to Claude Sonnet instead of Gemini:

```
# Telegram message examples:
"summarize this paper: arxiv.org/abs/2401.00000"
# -> routed to Gemini 2.5 Pro (default)

"@claude review the CVXPY solver logic in electoral/optimization/cvx.py"
# -> routed to Claude Sonnet
```

5. Security notes

The Pi should run OpenClaw inside a Docker container to sandbox file system access. Do not expose the OpenClaw gateway port to the public internet — use an SSH tunnel or a private Tailscale network instead. Restrict the Telegram/WhatsApp allowlist to your own number only. Run `openclaw doctor --fix` after any configuration change to catch misconfigurations before they create security gaps.

Recommended Daily Workflows

Task	Model	Example message
Summarize a paper	Gemini Pro	"Summarize arxiv.org/abs/X"
Morning project brief	Gemini Flash	Automated 8am cron
Quick code question	Gemini Pro	"What does XGBoost's `scale_pos_weight` do?"
Review pipeline logic	Claude	"@claude review kernels/shock.py"
Draft LaTeX paragraph	Claude	"@claude draft the methodology intro paragraph"
Debug test failure	Claude	"@claude debug this pytest output: [paste]"
Search for references	Gemini Pro	"Find 3 papers on NLP electoral forecasting"

B Target Repository Structure

```
electoral-equilibrium/
  README.md
  pyproject.toml
  AGENTS.md
  rawdata/                # ARDA, GSS, Gallup, NEP, Pew exports
  data/
    finetune/
      train.jsonl         # RoBERTa-scored + polling delta examples
      eval.jsonl          # held-out 2020 cycle
  configs/
    base.json
    smoke.json
    paper_baseline.json
```

```
    continuous.json        # pipeline_mode="continuous"; nightly incremental
        config
    shocks.json            # shock event registry with archive_ids field
    market_contracts.json  # static mapping: shock_id -> platform contract IDs
    bio_lexicon.json       # keyword patterns per bloc
  adapters/
    mistral-7b-electoral/  # saved QLoRA adapter weights (HPC output)
  artifacts/             # stage artifact JSON files
  electoral/
    cli.py
    pipeline.py          # DAG and stage ordering
    stages.py            # orchestration (thin wrappers)
    config.py            # PipelineConfig + derive_seed
    artifacts.py         # typed stage payload classes
    core/
      io.py              # Parquet + JSON artifact I/O (tabular stages use Parquet
          )
      schema.py          # validation helpers
      types.py           # Cycle, BlocId, Weight, ShockDelta, etc.
      rng.py             # deterministic RNG + sub-seed derivation
    kernels/             # stage internals (pure functions)
      data.py
      baseline.py
      sentiment.py       # RoBERTa pipeline + dataset builder
      finetune.py        # QLoRA training orchestration
      shock.py           # inference + pipeline integration
      optimize.py
      metrics_tables.py
    data/
      loaders.py
      cleaning.py
      panel.py
    models/
      ml_baseline.py     # moment estimation, GP classifier, LOCO validation
      regression.py      # RoBERTa baseline comparison
      bootstrap.py       # covariance bootstrap
    nlp/
      scraper.py         # news corpus (runs on Intel Mac)
      social.py          # Bluesky/Apify/Facebook/TruthSocial collectors
      bio_classifier.py  # keyword lexicon + embedding classifier (CPU fallback)
      scorer.py          # RoBERTa with bloc-weighted aggregation
      elasticity.py      # elasticity quantification + unified dataset assembly
    markets/
      collector.py       # Polymarket, PredictIt, Metaculus, Manifold, IEM
          collector
      aggregator.py      # volume-weighted multi-market price aggregation + live
          query
    llm/
      trainer.py         # QLoRA training loop (MLX local / HPC)
      inference.py       # constrained decoding via outlines
      eval.py            # held-out cycle evaluation
    api/
      shock_endpoint.py  # FastAPI /estimate endpoint
    portfolios/
      weights.py         # equal-weight, GMV, stochastic
      constraints.py
      cvx.py             # CVXPY formulations
    optimization/
      simplex.py         # simplex projection
```

```
    simulation/
      montecarlo.py       # Monte Carlo engine
    metrics/
      performance.py      # win probability, equilibrium stats
      tables.py           # manuscript-aligned table builders
    reporting/
      export.py           # CSV / LaTeX / JSON export
  webapp/                 # Next.js frontend (deployed to Vercel)
    app/
      page.tsx            # main shock simulator UI (SSE EventSource)
      dashboard/
        page.tsx          # analyst dashboard (password-protected)
      api/
        dashboard/
          auth/
            route.ts      # dashboard login/logout handler
    components/
      ShockInput.tsx       # party toggle, event textarea, intensity slider
      ShockNarrative.tsx   # plain-language prediction from LLM delta bins
      CoalitionChart.tsx   # unified: opaque prediction bars + translucent
         rebalance overlay
      WinGauge.tsx         # win-probability semicircle + market prior display
      CoverageMatrix.tsx   # bloc x cycle data coverage heatmap
      AuditLog.tsx         # queryable DuckDB table view
    lib/
      api.ts              # FastAPI client (blocking + SSE modes)
      types.ts            # TypeScript mirrors of backend schemas
      schemas.ts          # Zod runtime validation schemas
  deploy/
    modal_app.py          # Modal deployment: Mistral adapter on serverless GPU
    hpc_vllm.sh           # SLURM: vLLM on A100 with OpenAI-compatible API
    backend_router.py     # Routes to Modal or HPC via INFERENCE_BACKEND env var
  scripts/
    compile_hailo.py      # Export all-MiniLM to ONNX + compile HEF for Pi NPU
    pi_bio_server.py      # FastAPI bio classifier server running on Pi AI HAT+
    generate_synthetic.py # Gemini 2.5 Pro synthetic training data generation
    sample_archives.py    # Stratified sampling from raw archives (runs on HPC)
    clean_with_llm.py     # local open-weight model cleaning (M5)
    hpc_submit.sh         # SLURM fine-tuning job
    run_montecarlo_hpc.sh # SLURM Monte Carlo array job
    start_demo.sh         # Starts FastAPI + Next.js + ngrok for demo day
    audit_panel.py        # Data quality audit
    sensitivity_analysis.py
    verify_artifacts.py   # Replication verifier for paper baseline
  data/
    panel/
      voter_panel.parquet  # cleaned longitudinal voter panel (Parquet)
    embeddings/            # Parquet cache of RoBERTa embeddings per post
    archives/              # pre-collected historical tweet/social datasets
      kavanaugh/           # 56M tweets, Sept-Oct 2018 (Pushshift)
      election_2020_ieee/  # 3.5M tweets, July-Aug 2020 (IEEE DataPort)
      election_2020_kaggle/ # Oct 15-Nov 4 2020 (Kaggle)
      metoo/               # Oct 2017+ (Data.world)
      congress/            # ongoing JSON (GitHub alexlitel)
      polarization/        # Kiran et al.; ideologically-sorted bios
      smapp_elections/     # SMAPP NYU multi-cycle elections
      uw_researchworks/    # UW political dataset (verify schema)
      political_kaggle/    # kaushiksuresh147 political tweets
      partisanship/        # lingshuhu partisanship labels
```

```
      gender_political/    # lingshuhu political + gender labels
      telegram_2024/       # USC Telegram 2024 election (leonardo-blas)
      truth_social_2024/   # Truth Social 2024 election (kashish-s)
      tiktok_2024/         # TikTok 2024 election captions (gabbypinto)
      reddit/              # Pushshift Reddit archive (key subreddits, pre-2023)
      discord/             # Discord-Unveiled-Compressed (HF); politically
          relevant servers only
      x_2024/              # X 2024 election (sinking8; verify format first)
      echen_2020/          # US Pres Elections 2020 (echen102; verify format)
      bot_blocklist/       # khanradcoder bot IDs (filter only, not training)
      news/
        3dlnews/           # 3DLNews2: 8M+ URLs, 14k local outlets, 1995-2024 (W&M
            )
        webhose/           # Webhose free news dump (historical articles, full
            text)
      README.md            # schema notes per dataset, bio field availability
    finetune/
      train.jsonl          # real training examples
      synthetic.jsonl      # Gemini-generated examples (weight=0.5)
      eval.jsonl           # held-out 2020 cycle
    bio_labels/
      labeled_bios.jsonl   # manually labelled bios for SetFit training
    audit.duckdb           # persistent inference audit log
  tests/
    conftest.py
    fixtures/
      toy_panel.csv
      toy_config.json
    test_artifact_roundtrip.py
    test_rng.py
    test_schema.py
    test_data_cleaning.py
    test_baseline.py
    test_sentiment.py
    test_social.py
    test_bio_classifier.py
    test_llm_output_schema.py
    test_constrained_output.py
    test_endpoint_latency.py
    test_shock.py
    test_optimization.py
    test_montecarlo.py
    test_bootstrap.py
    test_metrics.py
    test_integration_end_to_end.py
    test_determinism_full_run.py
    test_artifact_schemas.py
    test_mc_convergence.py
```